

Dory: Streaming PCG with Small Memory

Xiaojie Guo^{*,†} Hanlin Liu[‡] Zhicong Huang[§] Hongrui Cui[¶] Wenhao Zhang[‡]
Cheng Hong[§] Xiao Wang[‡] Kang Yang[†] Yu Yu^{¶,*}

September 23, 2025

Abstract

Pseudorandom correlation generators (PCGs) have been popular in generating a huge amount of correlated randomness, a critical resource in secure computation. However, existing PCGs are memory-consuming and not friendly to resource-constrained devices. Even for moderate devices, the need for large memory can also be a disadvantage in applications like zero-knowledge proofs or large-scale secure computation. In this paper, we propose a malicious streaming PCG (sPCG), which generates a bounded number of tuples of subfield vector oblivious linear evaluation (sVOLE) on-the-fly, each with sublinear memory and computation.

1. We propose an efficient protocol that replaces the relaxed distributed comparison function in the best pseudorandom correlation function (PCF) for sVOLE (CRYPTO'22), which has the same streaming features for any polynomial number of tuples. With this protocol, our sPCG is doubly efficient in memory and the computation per sVOLE. Moreover, we augment the black-box distributed setup to malicious security and yield $4\times$ communication improvement. Our sPCG can be extended to a more efficient sVOLE PCF with the same improvements in memory and computation, and a $2\times$ faster malicious non-black-box distributed setup.
2. We present a practical attack on the Learning Parity with Noise (LPN) assumption for expand-accumulate codes with regular noise, revealing that some previous parameters provide around $14 \sim 22$ bits of security over binary noises, far below the target 128 bits. To address this, we introduce a low-Hamming-weight noise distribution to withstand the attack. We then derive some updated LPN parameters with the new noise distribution, restoring 128-bit security and reducing the noise-related computation and communication.
3. We provide an implementation of our sPCG for the special case of correlated oblivious transfer (COT). In addition to the improvements over the best PCF, our sPCG can have a comparable end-to-end performance to Ferret (CCS'20) and the PCG from expand-convolute codes (CRYPTO'23), two state-of-the-art PCGs, with the advantage of being able to produce 10 million COTs on-the-fly and reducing the memory from 337 MB and 624 MB to 20 MB, respectively.

1 Introduction

Pseudorandom correlation generators (PCGs) [BCGI18, BCG⁺19b, BCG⁺19a] play as a powerful primitive to distribute the correlated randomness consumed by secure multi-party computation

^{*}Shanghai Qi Zhi Institute, **Email:** guoxiaojie@sqz.ac.cn

[†]State Key Laboratory of Cryptology, **Email:** yangk@sklc.org

[‡]Northwestern University, **Email:** hanlin.liu@northwestern.edu, wenhao.zhang@northwestern.edu, wangxiao@northwestern.edu

[§]Ant Group, **Email:** zhicong.hzc@antgroup.com, vince.hc@antgroup.com

[¶]Shanghai Jiao Tong University, **Email:** rickfreeman@sjtu.edu.cn, yyuu@sjtu.edu.cn

(MPC) protocols. It gives multiple parties their respective short seeds, each of which can be locally expanded into a long string so that all strings meet an algebraic correlation. Based on the learning parity with noise (LPN) assumption, practical PCGs have been constructed, among which the PCG for subfield vector oblivious linear evaluation (sVOLE) [BCGI18, SGRR19, BCG⁺19b, BCG⁺19a, WYKW21, CRR21, GYW⁺23, RRT23] and its special case called correlated oblivious transfer (COT) [YWL⁺20] are of great importance. They can be used in generic MPC (e.g., [GMW87]), zero-knowledge proofs (e.g., [DIO21, BMRS21, YSWW21, WYKW21]), and private set intersection (e.g., [GPR⁺21, RS21]), etc. These applications can use PCG to achieve a highly efficient *silent preprocessing*, where the parties set up their PCG seeds with a small amount of communication in the offline phase and non-interactively expand them into long correlated strings to be consumed in the online phase.

An important limitation of PCG is its required runtime memory. In many PCG applications (e.g., zero-knowledge proofs [DIO21, BMRS21, YSWW21, WYKW21], neural network inference [HjLHD22], and multi-party triple generation [GYKW24]), the memory usage is a critical metric and can limit the scale of computation. For example, minimizing the memory usage is one design principle of the sVOLE-based zero-knowledge proofs [DIO21, BMRS21, YSWW21, WYKW21]; a smaller memory for sVOLE PCG immediately implies a smaller memory for these zero-knowledge proofs. In some scenarios, MPC runs among resource-constrained devices, such as mobile and edge devices, as well as web browsers (e.g., [DSZ14, ABL⁺18, BHKL18, BKRR25, JIF, WM]). In these devices, memory is a more scarce resource and can affect the user experience. Unfortunately, PCG cannot adequately meet the low-memory requirements in these settings. In general, the PCG expansion is monolithic as it outputs a huge number of correlations at once. These correlations are stored in the memory for a long time to be used across many protocols, or some of them will be wasted. Moreover, the memory consumption arises from the prior PCG constructions. To generate n sVOLE correlations, the PCGs from the primal-LPN assumption (e.g., [BCGI18, SGRR19, YWL⁺20, WYKW21]) use at least $\Omega(\sqrt{n})$ memory to store the pre-computed sVOLE correlations according to the primal-LPN parameters of [BCGI18]. In the PCGs based on the dual-LPN assumption (e.g., [BCG⁺19b, BCG⁺19a, GYW⁺23, RRT23]), $\Omega(n)$ memory is necessary to materialize the fully expanded shares of an LPN noise vector multiplied by a scalar¹.

To be memory-friendly, one could consider a *streaming* correlation generation: it outputs correlations *on-the-fly* so that they need not be stored, along with *sublinear* memory and computation per correlation (where the restriction on computation is for the non-trivial realization of low memory, see Footnote 1 for an example). Feasible results have been given by pseudorandom correlation functions (PCFs) [BCG⁺20a, OSY21, BCG⁺22, CD23, BCM⁺24], which extend the notion of PCGs in the spirit of the extension from pseudorandom generators (PRGs) to pseudorandom functions (PRFs). Given a PCF key and an on-the-fly public input, each party can locally compute one string assigned to the input and correlated with the other parties' strings. In particular, the state-of-the-art sVOLE PCF [BCG⁺22] in the minicrypt model can generate per correlation with $O(\log^2 n)$ memory and computation, where n is a *super-polynomial* bound on the number of correlations. It relies on the dual-LPN assumption and the expand-accumulate (EA) code. The super-polynomial n is not supported by PCGs, and this PCF should run generic secure two-party computation to set up PCF keys in a non-black-box manner to support it. When a fixed polynomial n suffices in some applications, [BCG⁺22] also propose a black-box setup protocol for PCF keys, but only with semi-honest security. Unfortunately in this case, the concrete efficiency of this PCF setup falls behind that of known PCGs.

¹If these expanded shares are not stored, they have to be recomputed in every inner product of the dual-LPN encoding, causing extra $\Omega(n)$ block-cipher calls per correlation.

Our contribution. In this work, we design new malicious protocols for the *streaming PCG* and PCF of subfield vector oblivious linear evaluation (sVOLE). The streaming PCG refers to such a relaxed form of PCF that the bound n of the number of generated correlations is (i) fixed *a priori* and polynomial, or (ii) virtually unbounded at the cost of a routine interactive bootstrapping. We summarize our results below and provide a detailed comparison in Table 1.

1. We propose an efficient protocol to replace the relaxed distributed comparison function, a core primitive of the state-of-the-art PCF [BCG⁺22], and use it in our construction of streaming PCG and PCF. It adopts an accumulation structure in binary tree to realize the same functionality and $O(\log N)$ computation. Compared to the previous PCF, we achieve halved memory and computation per generation in both streaming PCG and PCF. This protocol also improves the setup communication of the two constructions by a factor of $4\times$ and $2\times$, respectively.
2. We present a practical attack against the parameters of the LPN assumption with the EA code and regular noises, which underlie the state-of-the-art PCF and our constructions. When the regular noise is binary, this attack reduces the previously claimed 128-bit security to $14 \sim 22$ bits. We also propose a countermeasure to fill this gap and preserve the high efficiency of the instantiated constructions.
3. We implement our streaming PCG under the COT case, which is of particular interest for silent OT extension in generic MPC. We show that, to produce 10 million COTs, our streaming PCG uses only 20 MB memory, improving the memory usage of the primal-LPN PCG [YWL⁺20] and the dual-LPN PCG from the expand-convolute (EC) code [RRT23] by $16.7\times$ and $31.2\times$, respectively. The computation of our streaming PCG is about $2\times$ larger, leading to a memory-computation trade-off.

2 Technical Overview

2.1 Recap of sVOLE PCF in [BCG⁺22]

We first recall the state-of-the-art two-party PCF construction [BCG⁺22] for sVOLE correlations. In this construction, two parties can produce each sVOLE correlation $M(r) = K(r) + r \cdot \Delta$ on-the-fly, where the sender holds $(\Delta, K(r)) \in \mathbb{K}^2$ and the receiver holds $(r, M(r)) \in \mathbb{F} \times \mathbb{K}$ for an extension field $\mathbb{K}$ of field $\mathbb{F}$. The two parties begins with a *subtractive* sharing $\llbracket e \cdot \Delta \rrbracket$ for a global key Δ of the sender and a sparse vector $e \in \mathbb{F}^N$ of the receiver such that

$$\llbracket e \cdot \Delta \rrbracket_1 - \llbracket e \cdot \Delta \rrbracket_0 = e \cdot \Delta.$$

Then, upon a new request of a fresh sVOLE correlation, the two parties use the same randomness to sample a fresh public vector $\mathbf{b} \leftarrow \mathbb{F}^N$ and, given some public matrix $\mathbf{A} \in \mathbb{F}^{N \times N}$, define $r := \mathbf{b}^\top \mathbf{A} e \in \mathbb{F}$ and

$$K(r) := \mathbf{b}^\top \mathbf{A} \llbracket e \cdot \Delta \rrbracket_0 \in \mathbb{K}, \quad M(r) := \mathbf{b}^\top \mathbf{A} \llbracket e \cdot \Delta \rrbracket_1 \in \mathbb{K}.$$

When $n < N$ sVOLE correlations are produced on-the-fly, n sampled $\mathbf{b}$'s form the rows of a matrix $\mathbf{B} \in \mathbb{F}^{n \times N}$ while the sharing of $\llbracket e \cdot \Delta \rrbracket$ is reused for $\mathbf{r} := \mathbf{B} \mathbf{A} e \in \mathbb{F}^n$. The work showed that, if all $\mathbf{b}$ vectors and e are sparse, and $\mathbf{A}$ is the accumulator matrix with 1's on its lower triangle and 0's otherwise, $\mathbf{r}$ can be pseudorandom under the dual-LPN assumption with a public compressing matrix $\mathbf{H} = \mathbf{B} \mathbf{A}$. This assumption is called EA-LPN assumption.

		[BCGI18, SGRR19, YWL+20, WYKW21]	[RRT23]	[BCG+22]	This work
Assumption		primal-LPN (d -local linear code)	dual-LPN (EC code)	dual-LPN (EA code ^a)	dual-LPN (EA code)
Streaming properties	Is on-the-fly?	✗	✗	✓	✓
	Memory (bits)	$\Omega(\lambda\sqrt{n})$	$\Omega(\lambda N)$	$2t\lambda \log \frac{N}{t}$	$t\lambda \log \frac{N}{t}$
	Comp. per corr.	$O(d)$ RMA + $O(1)$ BCs + $O(d)$ field mult.	$O(\ell)$ RMA + $O(1)$ BCs + $O(\log N)$ field mult.	$2\ell \log \frac{N}{t}$ BCs + ℓ field mult.	$\ell \log \frac{N}{t}$ BCs + ℓ field mult.
Black-box Distributed Setup (when $n = \text{poly}(\lambda)$ or $N = \text{poly}(\lambda)$)					
Security		Malicious	Malicious	Semi-honest	Malicious
Comp. (BCs)		$2n$	$2N$	$3N$	$2N^b$
Comm. (bits)	R $\rightarrow$ S	$t \log \mathbb{F} $	$t \log \mathbb{F} $	$t \log \mathbb{F} $	$t \log \frac{N}{t} \log \mathbb{F} $
	S $\rightarrow$ R	$2t\lambda \log \frac{n}{t}$	$2t\lambda \log \frac{N}{t}$	$4t\lambda \log \frac{N}{t}$	$t\lambda \log \frac{N}{t}$
Pre-computation		$(\Omega(\sqrt{n}) + t)$ sVOLEs + $t \log \frac{n}{t}$ OTs	t sVOLEs + $t \log \frac{N}{t}$ OTs	t sVOLEs + $t \log \frac{N}{t}$ OTs	$t(\log \frac{N}{t} + 1)$ sVOLEs

^a They do not use LPN noises with a doubled weight t as we do in the COT case. Such noises are vulnerable to our attacks in Section 5.1.

^b For the semi-honest security, both parties use N block-cipher calls. Otherwise, for $\mathbb{F} = \mathbb{F}_2$, the receiver can also use N block-cipher calls.

Table 1: Comparison to the state-of-the-art generation of n sVOLE tuples, where $\mathbb{F}$ (resp., $\mathbb{K}$) denotes the field (resp., extension field) and $|\mathbb{K}| \approx 2^\lambda$. “S” and “R” represents the sender and the receiver, respectively. All assume LPN noises with t regular positions. In the dual-LPN assumption, $N = s \cdot n$ for some constant $s > 1$. For the EC code and EA code, $\ell = O(\log N)$ denotes the row weight of the expander matrix. “BCs” refers to the number of block-cipher calls and “RMA” refers to the random memory access to the used memory. “Memory” excludes the $O(n)$ storage cost of generated tuples and this cost occurs only if the “Is on-the-fly?” is not true. Some low-order terms are omitted for simplicity.

In this construction, an important observation is that the two parties need not compute explicit matrix-vector multiplications in the above LPN encoding. Given accumulator matrix $\mathbf{A}$ and the $t = O(\log N)$ sparsity of $\mathbf{e} \cdot \Delta$ such that $\mathbf{e}^{(\alpha_i)} = \beta_i \neq 0$ for each $i \in [t]$ and $\mathbf{e}^{(i)} = 0$ otherwise,

$$\mathbf{A} \mathbf{e} \cdot \Delta = \sum_{i \in [t]} (0, \dots, 0, \underbrace{\beta_i}_{\text{at index } \alpha_i}, \dots, \beta_i)^\top \cdot \Delta.$$

Thus, for any ℓ -sparse row vector $\mathbf{b}^\top$ such that $\mathbf{b}^{(j)} \neq 0$ if and only if $j \in \mathcal{J} \subseteq [N]$ for $|\mathcal{J}| = \ell$, the inner product between $\mathbf{b}^\top$ and $\mathbf{A} \mathbf{e} \cdot \Delta$ is equivalent to the following sum of some comparison function results:

$$\begin{aligned} \mathbf{b}^\top \mathbf{A} \mathbf{e} \cdot \Delta &= \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \mathbb{1}(j \geq \alpha_i) \cdot \beta_i \cdot \Delta \\ &= \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot f_{\alpha_i \in [N], \beta_i \cdot \Delta \in \mathbb{K}}^{\geq}(j). \end{aligned} \tag{1}$$

The PCF suggested the use of regular noise $\mathbf{e}$, where there are t blocks of size $\frac{N}{t}$ and only one entry in each block is non-zero. With regular noises, (1) can be written as the sum of *smaller-domain* comparison functions as follows:

$$\mathbf{b}^\top \mathbf{A} \mathbf{e} \cdot \Delta = \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \begin{cases} 0, & \mathcal{L}(j) < i \\ f_{\mathcal{I}(\alpha_i) \in [\frac{N}{t}], \beta_i \cdot \Delta}^{\geq}(\mathcal{I}(j)), & \mathcal{L}(j) = i \\ \beta_i \cdot \Delta, & \mathcal{L}(j) > i \end{cases} \tag{2}$$

where $\mathcal{L} : [N] \rightarrow [t]$ (resp., $\mathcal{I} : [N] \rightarrow [\frac{N}{t}]$) is a public function that maps an index to its block index (resp., inner-block index) as per regular noises. If one can choose ℓ sublinear in N and the evaluation of the comparison functions is sublinear-time, the computation in (2) can grow sublinearly in N , in contrast to the heavy vector-matrix multiplication $\mathbf{b}^\top \mathbf{A} \llbracket \mathbf{e} \cdot \Delta \rrbracket$ for $\mathbf{K}(r)$ and $\mathbf{M}(r)$.

Finally, the work showed how the two parties can subtractively share $\llbracket \mathbf{b}^\top \mathbf{A} \mathbf{e} \cdot \Delta \rrbracket$ for many fresh $\mathbf{b}$'s but the same noise $\mathbf{e}$. It noticed that the comparison functions in (2) can be shared between the two parties using *relaxed distributed comparison functions (RDCFs)*, which relaxes the security of distributed comparison functions [BGI19, BCG⁺21] by allowing the receiver to learn all α_i 's. An RDCF scheme distributes a key pair between the two parties to subtractively share the full-domain evaluation of a comparison function $f_{\alpha, \beta, \Delta}^{\geq}(x)$, where only the shares on a few public inputs (such as all $\mathcal{I}(j)$'s of different public $\mathbf{b}$'s) are retrieved. The RDCF construction in [BCG⁺22] uses a GGM-style ternary tree, which ensures that the evaluation on an input is $O(\log N)$. Each party holds a ternary tree, and its shares of the full-domain evaluation are the leaf nodes corrected by the other nodes in its ternary tree. This ternary tree lacks any algebraic structure and incurs a major performance bottleneck. Moreover, it is unknown how to distribute such a tree between the two parties with malicious security and a black-box setup [Ds17] when N can be polynomial.

2.2 Our Streaming PCG for sVOLE

We briefly introduce the intuition behind our malicious streaming PCG protocol for sVOLE. We point out that an RDCF is an overkill to share the point-wise evaluation of a comparison function when α is known to the receiver. Mathematically, a comparison function can be essentially reduced to a point function as follows:

$$\begin{aligned} f_{\alpha, \beta, \Delta}^{\geq}(x) &= \mathbb{1}(x \geq \alpha) \cdot \beta \cdot \Delta \\ &= \sum_{i \leq x} \mathbb{1}(i = \alpha) \cdot \beta \cdot \Delta = \sum_{i \leq x} f_{\alpha, \beta, \Delta}^{\bullet}(i), \end{aligned} \tag{3}$$

where $f_{\alpha, \beta, \Delta}^{\bullet}$ is a point function evaluated to $\beta \cdot \Delta$ if the input equals α and 0 otherwise, and the second equality comes from that $x \geq \alpha$ if and only if any value not greater than x equals α . This equality allows us to turn the evaluation of $f_{\alpha, \beta, \Delta}^{\geq}(x)$ into many evaluations of $f_{\alpha, \beta, \Delta}^{\bullet}(i)$. Similar to RDCFs, we need not hide α from the receiver. Thus, it is natural to expect that the two parties subtractively share the full-domain evaluation of the point function via a primitive “relaxed distributed point function” and retrieve their shares on some inputs. In fact, this primitive can be realized by a puncturable pseudorandom function (PPRF) [SGRR19, BCG⁺19a, GYW⁺23]. Roughly speaking, the sender samples a PRF key k and evaluates the PRF F on the full domain as its shares of the full-domain evaluation of $f_{\alpha, \beta, \Delta}^{\bullet}(i)$; the receiver with α can run with the sender to get a PPRF key k_α , which can be used to evaluate F on the full domain except α , and a share $F_k(\alpha) + \beta \cdot \Delta$ on the input α .

Although this construction allows the two parties to share the full-domain evaluation of $f_{\alpha, \beta, \Delta}^{\bullet}(i)$, each party needs $O(N)$ computation on average to obtain its share of $\sum_{i \leq x} f_{\alpha, \beta, \Delta}^{\bullet}(i)$ since its share of each $f_{\alpha, \beta, \Delta}^{\bullet}(i)$ contains a PRF output and should be accumulated one-by-one. This overhead violates the sublinear computation required by a streaming PCG.

Efficient accumulation. To use the observation in (3) to build a streaming PCG, we require a sublinear-time accumulation of $\sum_{i \leq x} f_{\alpha, \beta, \Delta}^{\bullet}(i)$. We observe that an $O(\log N)$ accumulation can be achieved in a binary tree with the following property: the sum of all leaf nodes of any sub-tree equals the root of this sub-tree. This property can be fulfilled using the invariant of a correlated GGM (cGGM) tree [GYW⁺23], where the left child of a node v equals its hash value $\mathbf{H}(v)$ and its right child is defined as $v - \mathbf{H}(v)$.

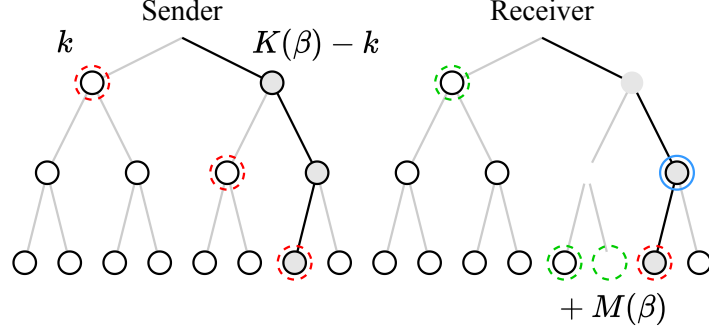


Figure 1: The accumulation of the two parties when $\alpha = (101)_2$ (known to the receiver) and $x = (110)_2$. The nodes to be accumulated are marked by dashed circles. For the receiver, the sum of three green dashed nodes can be replaced by the negative blue node due to the patch of the α -th leaf node (before adding $M(\beta)$ to it) and the cGGM invariant.

More specifically, the sender sets two first-level nodes as $(k, \Gamma - k)$ and recursively uses the cGGM invariant to expand a binary tree, whose leaf nodes are its shares of the full-domain evaluation of $f_{\alpha, \beta \cdot \Delta}^\bullet(i)$. To compute its shares of $\sum_{i \leq x} f_{\alpha, \beta \cdot \Delta}^\bullet(i)$, the sender accumulates the leaf node labeled by x and the at most $\log N$ nodes left adjacent to the path x . On the other side, before the accumulation, the receiver receives some messages from the sender to obtain the sum of either left or right nodes for each level of the sender's tree. The receiver can virtually recover all leaf nodes except that for α and *locally* patch this node as the negative sum of all other leaf nodes. Here, the leaf nodes of the sender and the receiver form the *subtractive* shares of $(0, \dots, 0, -\Gamma, 0, \dots, 0)$. If the sender sets $\Gamma = K(\beta)$ and the receiver further adds $M(\beta)$ to its α -th leaf node, where $M(\beta) = K(\beta) + \beta \cdot \Delta$ is a pre-computed sVOLE, the two parties can obtain the subtractive shares of

$$(0, \dots, 0, \beta \cdot \Delta, \dots, 0), \quad (4)$$

which is the full-domain evaluation of $f_{\alpha, \beta \cdot \Delta}^\bullet(i)$. Then, we design the accumulation of the receiver as follows: the receiver tries to accumulate all left neighbor nodes of the path x . If there is one (and at most one) left neighbor node on the path α , the receiver computes this node as the sum of all leaf nodes in the sub-tree rooted in this node. Recall that the punctured point α lies in this sub-tree. The patch of α and the cGGM invariant allows us to compute this sum from the nodes adjacent to the path α , which are already computed in the interaction. Our accumulation for the receiver runs in $O(\log N)$ time. The correctness of this accumulation follows from the cGGM invariant.

In Figure 1, we demonstrate how each party computes its share of $\sum_{i \leq x} f_{\alpha, \beta}^\bullet(i)$ via our accumulation algorithms in one top-down pass of the cGGM tree.

Malicious security. To ensure that the non-interactive expansion can be simulated against a malicious adversary, we require that the interactive setup of our streaming PCG is maliciously secure. In our construction, we adapt the consistency check [YWL⁺20, WYKW21] and the programming-based security of cGGM tree [GYW⁺23].

Recall that the malicious sender can send malformed messages to the honest receiver to inject additive errors to the per-level sums of left or right nodes of the cGGM tree. To prevent such errors, the two parties runs a consistency check where each party computes a random linear universal hash of its all leaf nodes. If the two hash values are not consistent with the result from applying this hash to the full-domain evaluation of $f_{\alpha, \beta \cdot \Delta}^\bullet(i)$, i.e., (4), the receiver will abort. Since this check

depends on the secret (α, β) (i.e., the LPN noise) of the receiver, the malicious sender can launch a selective failure attack against the LPN noise, which underlies the pseudorandomness of $\mathbf{r} := \mathbf{B}\mathbf{A}\mathbf{e}$. To preserve this pseudorandomness, we need the LPN assumption with static leakage of the LPN noise.

A subtle issue for this check is the sublinear restriction on the memory. In our protocol, the linear universal hash function should be sampled at random in the consistency check, i.e., after the on-the-fly cGGM-tree expansion. For the two parties, the sublinear memory prevents them from materializing all leaf nodes so that they need to recompute their cGGM trees on-the-fly to incrementally compute the hash of all leaf nodes. As a result, the total computation per party is $2N$ block-cipher calls, instead of N with no memory restriction or in semi-honest security.

In particular, the malicious receiver can use the cGGM-tree messages and check messages from the sender to test their consistency between the sVOLE global key Δ . More specifically, these messages are simulated by random values in the ideal world and are not as consistent with Δ as in the real world, failing the security proof. This challenge can be addressed in the ideal model by allowing the malicious receiver to guess the sVOLE global key and programming the ideal model for a correctly guessed Δ to maintain the consistency between these messages and Δ . When $\mathbb{F} \neq \mathbb{F}_2$, the receiver can also send a malformed message to choose a *non-binary* “path” α_i of the cGGM tree, making the extraction of the simulator failed. In this case, we use equality test and linear universal hashing to have the honest sender abort in the consistency check since such a malicious receiver cannot pass the check with a noticeable probability. When $\mathbb{F} = \mathbb{F}_2$, the one-side check [YWL⁺20] (where the linear universal hash can be chosen by the receiver so that it can apply this hash to its tree incrementally, without recomputing it) plus the above programming suffices since α_i must be binary.

2.3 Our PCF for sVOLE

The cGGM-based accumulation in our streaming PCG can be naturally extended to a super-polynomial bound N since it still has a polynomial complexity $O(\log N)$. As a result, we can construct an sVOLE PCF whose evaluation algorithm is identical to the evaluation in our streaming PCG. For each party, the PCF key is essentially defined as its seed of the streaming PCG. However, for the distributed PCF key generation (i.e., without a trusted dealer), a super-polynomial N excludes our black-box PCF key generation protocol where a binary tree with N leaf nodes is fully expanded. Instead, the two parties can use generic secure two-party computation (2PC) to run the PCF key generation algorithm. This protocol is *non-black-box* since it requires the circuit of the hash function in the cGGM invariant.

Consider the generation of COT tuples for example. In the prior PCF [BCG⁺22], the tree expansion in the RDCF uses two length-preserving PRG calls per level, where each PRG call costs a call to the fixed-key block cipher (e.g., AES). In contrast, the tree expansion in our PCF performs one hash call per level, and this hash can be implemented with one call to the fixed-key block cipher (see [GKWY20, GYW⁺23]). This leads to $2\times$ improvement in the overall circuit complexity, implying the same improvement in both the computation and communication of the 2PC-based PCF key generation. A similar improvement holds for general sVOLE tuples.

2.4 Cryptanalysis against EA Codes

We observe that each row of $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell, \mathcal{R})$ has $\ell + 1$ consecutive blocks, each filled with a fixed value, and the inner product between such a block and a regular noise block (with Hamming weight 1) reduces to multiplying a public value with an unknown non-zero variable. In particular,

for $\mathcal{R} = \mathbb{F}_2$, the only possible unknown non-zero variable is 1. Hence, for any public $a \in \mathbb{F}_2$, the inner product between a regular noise of Hamming weight t and a block filled with a is simply $a \cdot t \bmod 2$, which does not require any additional variable.

Following from the above observation, we propose two attacks: a recovery attack and a distinguishing attack.

1. Our recovery attack targets the EA-LPN assumption with regular noise and is particularly effective for $\mathcal{R} = \mathbb{F}_2$. Using our observation, the inner product between $\ell + 1$ identical blocks and a weight- t regular noise reduces to a lower-dimensional one, leading to a smaller dual-LPN instance to be solved.
2. Our distinguishing attack only applies to the EA-LPN assumption over $\mathbb{F}_2$ with regular noise. Each row of $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell, \mathbb{F}_2)$ has $\ell + 1$ consecutive blocks of all 0s or all 1s, while the regular noise vector $\mathbf{e}$ consists of t equal-length blocks, each with exactly a single 1 and all other entries set to 0. Therefore, each bit of $\mathbf{b} = \mathbf{H} \cdot \mathbf{e}$ is influenced by at most ℓ entries in $\mathbf{e}$, making it predictable with a probability of roughly $1/2 + (1/2)^\ell$, and yielding a bit security of roughly 2ℓ (usually $\ell \in \{7, 9, 11\}$).

To resist our attacks, we modify the regular noise distribution over $\mathbb{F}$: in each block, one random position is set to a uniform element in $\mathbb{F}$, and all others are 0. We refer to this noise as *the relaxed regular noise*. We note that the relaxed regular noise resists our attacks because each value in $\mathbf{b}$ is affected by the majority of noise blocks, rather than just ℓ of them. In particular, the inner product between a fixed-value block and a noise block with a uniform element can introduce sufficient randomness, which is further enhanced by the large minimum distance of the EA code.

To use this noise and resist other attacks, such as algebraic attacks [BØ23] and information set decoding (ISD) [MMT11, BJMM12, LWYY24, ES24, KL24], we will increase the number of noise blocks such that the total Hamming weight equals (in expectation) the original one of regular noise. Although each block contains only one uniform element, this adjustment increases the randomness of $\mathbf{b}$ and makes it more difficult for attackers to exploit the sparsity or regular structure.

3 Preliminaries

3.1 Notation

Let λ denote the computational security parameter and $\log(\cdot)$ to denote logarithms in base 2. We define $[a, b) = \{a, \dots, b - 1\}$ and write $[a, b] = \{a, \dots, b\}$ and $[a] = [0, a - 1]$. We use $x \leftarrow S$ to denote sampling x uniformly at random from a finite set S . We use $\{x_i\}_{i \in S}$ to denote the set collecting all elements with indices in set S . When the context is clear, we abuse the notation and use $\{x_i\}$ to denote such a set. We use bold lowercase letters (e.g., $\mathbf{a}$) for column vectors and bold uppercase letters (e.g., $\mathbf{A}$) for matrices. For any $i, j \geq 0$, let $\mathbf{a}^{(i)}$ be the i -th entry of $\mathbf{a}$, $\mathbf{A}^{(i,j)}$ be the entry on the i -th row and j -th column of $\mathbf{A}$, $\mathbf{A}^{(i,\cdot)}$ be the i -th row of $\mathbf{A}$, and $\mathbf{A}^{(\cdot,j)}$ be the j -th column. For any Boolean string $\alpha = \alpha_0 \dots \alpha_{n-1} \in \{0, 1\}^n$ and $0 \leq i \leq n$, we write $\alpha|_i = \alpha_0 \dots \alpha_{i-1}$ to denote its i -bit prefix (where $\alpha|_0$ is 0 for completeness).

3.2 Learning Parity with Noise and EA Codes

We adapt the previous definition of dual-LPN assumption with static leakage [BCG⁺22] to the noises having t regular positions but uniform payloads, instead of weight- t regular noises (where the payloads are uniform but non-zero). This adaption is for the noise resisting our attack in Section 5.1. We call this new noise *the relaxed regular noise*.

Definition 1 (Dual-LPN [BCG⁺19b]). Given a ring $\mathcal{R}$, $n = n(\lambda)$, $N = N(\lambda) \geq n$, and $t = t(\lambda) \leq N$, let $\mathcal{D} = \mathcal{D}(t, N, \mathcal{R})$ be the relaxed regular noise distribution on $\mathcal{R}^N$ and $\mathcal{C} = \mathcal{C}(n, N, \mathcal{R})$ be a probabilistic code generation algorithm sampling a matrix $\mathbf{H} \leftarrow \mathcal{R}^{n \times N}$. The dual-LPN assumption $\text{dLPN}(\mathcal{D}, \mathcal{C}, \mathcal{R}, t, n, N)$ says that, for any PPT adversary $\mathcal{A}$,

$$\{(\mathbf{H}, \mathbf{b}) \mid \mathbf{H} \leftarrow \mathcal{C}, \mathbf{e} \leftarrow \mathcal{D}, \mathbf{b} := \mathbf{H}\mathbf{e}\} \stackrel{c}{\approx} \{(\mathbf{H}, \mathbf{b}) \mid \mathbf{H} \leftarrow \mathcal{C}, \mathbf{b} \leftarrow \mathcal{R}^n\}$$

Definition 2 (Dual-LPN with static leakage [BCG⁺19a]). Given a ring $\mathcal{R}$, $n = n(\lambda)$, $N = N(\lambda) \geq n$, and $t = t(\lambda) \leq N$, let $\mathcal{D} = \mathcal{D}(t, N, \mathcal{R})$ be the relaxed regular noise distribution on $\mathcal{R}^N$ and $\mathcal{C} = \mathcal{C}(n, N, \mathcal{R})$ be a probabilistic code generation algorithm sampling a matrix $\mathbf{H} \leftarrow \mathcal{R}^{n \times N}$. The dual-LPN assumption with static leakage $\text{dLPN-SL}(\mathcal{D}, \mathcal{C}, \mathcal{R}, t, n, N)$ says that, for any PPT adversary $\mathcal{A}$,

$$\left| \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{D}, \mathcal{C}, \mathcal{R}, t, n, N}^{0, \text{dLPN-SL}}(\lambda) = 1 \right] - \Pr \left[\text{Exp}_{\mathcal{A}, \mathcal{D}, \mathcal{C}, \mathcal{R}, t, n, N}^{1, \text{dLPN-SL}}(\lambda) = 1 \right] \right| \leq \text{negl}(\lambda),$$

where the experiment $\text{Exp}_{\mathcal{A}, \mathcal{D}, \mathcal{C}, \mathcal{R}, t, n, N}^{b, \text{dLPN-SL}}(\lambda)$ for each $b \in \{0, 1\}$ is defined as follows:

1. Sample $\mathbf{e} \leftarrow \mathcal{D}$ and $\mathbf{H} \leftarrow \mathcal{C}$. Let $\alpha_0, \dots, \alpha_{t-1} \in [N]$ denote the sorted indices of the regular positions in $\mathbf{e}$, i.e., $\alpha_i \in [i \cdot \frac{N}{t}, (i+1) \cdot \frac{N}{t}]$ for each $i \in [t]$.
2. $\mathcal{A}$ sends t sets $\mathcal{I} := (I_0, \dots, I_{t-1}) \subseteq [N]^t$.
3. If $\alpha_i \in I_i$ holds for each $i \in [t]$, send success to $\mathcal{A}$; otherwise, send fail to $\mathcal{A}$.
4. If $b = 1$, compute $\mathbf{b} := \mathbf{H}\mathbf{e} \in \mathcal{R}^n$; otherwise, sample $\mathbf{b} \leftarrow \mathcal{R}^n$. Send $(\mathbf{H}, \mathbf{b})$ to $\mathcal{A}$.
5. Receive a bit b' from $\mathcal{A}$ and output b' .

For the concrete efficiency, we focus on the following definition of EA code with a weight- ℓ noise distribution for each row of the expander matrix.

Definition 3 (Expand-accumulate code [BCG⁺22]). Given a ring $\mathcal{R}$, $n = n(\lambda)$, $N = N(\lambda) \geq n$, and $\ell = \ell(\lambda) \leq N$, let $\mathcal{B} = \mathcal{B}(\ell, N, \mathcal{R})$ be the weight- ℓ noise distribution on $\mathcal{R}^N$. The code generation algorithm $\text{EAGen}(n, N, \ell, \mathcal{R})$ for the expand-accumulate code is as follows:

- Sample vectors $\mathbf{b}_0, \dots, \mathbf{b}_{n-1} \leftarrow \mathcal{B}$ independently.
- Output $\mathbf{H} = \mathbf{B}\mathbf{A}$, where $\mathbf{A} \in \mathcal{R}^{N \times N}$ is an accumulator matrix and the expander matrix $\mathbf{B} \in \mathcal{R}^{n \times N}$ is the concatenation of the sampled vectors, i.e.,

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{b}_0^\top \\ \vdots \\ \mathbf{b}_{n-1}^\top \end{bmatrix}.$$

We will omit $\mathcal{R}$ in the notation if $\mathcal{R} = \mathbb{F}_2$.

In this paper, we define the dual-LPN assumption with the EA code (or the EA-LPN assumption in short) as the dual-LPN assumption where $\mathcal{C} = \text{EAGen}$. We write the EA-LPN assumption as $\text{EA-LPN}(t, n, N, \ell, \mathcal{R})$. Similarly, we define the EA-LPN assumption with static leakage as $\text{EA-LPN-SL}(t, n, N, \ell, \mathcal{R})$. In the EA-LPN assumption with static leakage, the adversary can perform a selective failure attack on the noise $\mathbf{e}$, resulting in (on-average) one-bit leakage. Prior attacks cannot significantly reduce their cost using this leakage. Similar LPN assumptions have already been used in many PCG works (e.g., [BCG⁺19a, YWL⁺20, WYKW21, BCG⁺20b]).

Functionality $\mathcal{F}_{\text{sVOLE}}$

This functionality runs with sender P_0 and receiver P_1 .

Parameters: Field $\mathbb{F}$ and its extension $\mathbb{K}$.

Initialize: This interface is called only once and ahead of other interfaces. Upon receiving (init) from both parties,

1. If P_0 is corrupted, receive $\Delta \in \mathbb{K}$ from the adversary; otherwise, sample $\Delta \leftarrow \mathbb{K}$. Store Δ and output Δ to P_0 .

Extend: Upon receiving (extend, m) from both parties:

1. If P_0 is corrupted, receive $\mathbf{v} \in \mathbb{K}^m$ from the adversary; otherwise, sample $\mathbf{v} \leftarrow \mathbb{K}^m$.
2. If P_1 is corrupted, receive $(\mathbf{u}, \mathbf{w}) \in \mathbb{F}^m \times \mathbb{K}^m$ from the adversary, and $\mathbf{v} := \mathbf{w} - \mathbf{u} \cdot \Delta \in \mathbb{K}^m$; otherwise, sample $\mathbf{u} \leftarrow \mathbb{F}^m$ and $\mathbf{w} := \mathbf{v} + \mathbf{u} \cdot \Delta \in \mathbb{K}^m$.
3. Output $\mathbf{v}$ to P_0 and $(\mathbf{u}, \mathbf{w})$ to P_1 .

Global-key queries: When P_1 is corrupted, upon receiving (guess, Δ') from the adversary, send (success) to the adversary if $\Delta = \Delta'$; send (fail) to the adversary otherwise.

Figure 2: Functionality for subfield VOLE.

3.3 Security Model and Functionality

We adopt the *universal composability* (UC) framework [Can01] to prove security in the presence of a malicious, static adversary. We say that a protocol Π *UC-realizes* an ideal functionality $\mathcal{F}$ if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, there exists a PPT adversary (simulator) $\mathcal{S}$ such that for any PPT environment $\mathcal{Z}$ with arbitrary auxiliary input z , the output distribution of $\mathcal{Z}$ in the *real-world* execution where the parties interact with $\mathcal{A}$ and execute Π is computationally indistinguishable from the output distribution of $\mathcal{Z}$ in the *ideal-world* execution where the parties interact with $\mathcal{S}$ and $\mathcal{F}$.

Our protocols use the functionality $\mathcal{F}_{\text{sVOLE}}$ (in Fig. 2) of subfield vector oblivious linear evaluation as a subroutine. We omit the session IDs and sub-session IDs in the functionalities for simplicity. By convention, we can write sVOLE tuples as two-party information-theoretic message authentication codes (IT-MACs) [NNOB12, DPSZ12]. Let $\Delta \in \mathbb{K}$ denote the global key of the sender P_0 . P_0 authenticates a value $x \in \mathbb{F}$ of the receiver P_1 by sampling a uniform one-time key $K(x) \leftarrow \mathbb{K}$ and giving to P_1 the MAC value $M(x) := K(x) + x \cdot \Delta \in \mathbb{K}$. We also use two standard and basic functionalities: (i) $\mathcal{F}_{\text{EQ}}$ of equality test [LLY⁺25], which outputs a bit to two parties to show whether their inputs are identical or not, and (ii) $\mathcal{F}_{\text{Rand}}$ of coin-tossing [GYW⁺23], which outputs the same random string to two parties and can be realized using Fiat-Shamir heuristic [FS87].

4 Streaming Generation of sVOLE

In this section, we detail our construction of the streaming PCG and PCF for sVOLE correlations. As shown in Section 2.2, this PCG can be obtained from combining the EA-LPN assumption and a relaxed distributed comparison function, which is equivalent to computing the partial sum of a “relaxed distributed point function”. We show how to implement this function with a correlated GGM (cGGM) tree [GYW⁺23] and using its internal structure to realize an efficient accumulation

of the partial sum. The implementation and accumulation underlie our streaming PCG for sVOLE and can be extended to our PCF construction.

4.1 Accumulation in Correlated GGM Trees

In a cGGM tree, for a node x , its left and right child nodes are $H(x)$ and $x - H(x)$ respectively. This invariant implies that for each sub-tree T with root x in a cGGM tree, the sum of all leaf nodes in T equals to x . Using this corollary, the streaming feature of our PCG is realized from the streaming accumulation algorithms on cGGM trees as shown in Fig. 3. We explain the accumulation of the sender first, followed by that of the receiver.

The sender side. Given an index ω , the accumulation is to compute the sum of all leaf nodes with indices not greater than ω . These leaf nodes form a consecutive block starting from the first leaf node. Thus, they can be partitioned into multiple smaller subsets, where each set forms *all the leaf nodes* of a smaller-sized full binary tree. Then, by utilizing the corollary in a cGGM tree, the sum of all leaf nodes of a sub-tree equals to their common root. Thus, the sum of all leaf nodes with index smaller than ω equals to the sum of a series of internal nodes, which are the root of the sub-trees.

These internal nodes form the left sibling nodes from the path indexed by ω , as shown by the red dashed nodes in Fig. 1. Hence, the accumulation algorithm of the sender P_0 proceeds as follows. First, P_0 initializes an accumulator to zero. Then starting from the first level, P_0 computes the nodes on the path ω as well as the sibling nodes, and update the accumulator with these left sibling nodes. Finally, P_0 updates the accumulator with the leaf node indexed by ω . From the above argument, the resulting accumulator is the sum of all leaf nodes with indices not greater than ω .

The receiver side. The receiver P_1 only learns the sibling nodes along the path α of P_0 's cGGM tree and its α -th leaf node is computed differently. Therefore, the receiver may not run the accumulation of the sender. We design a different algorithm `cGGM.PuncSetup` to handle its accumulation and return a punctured key k_α containing the sibling nodes along the path α and the alternatively defined α -th leaf node.

The case of $\omega = \alpha$ is simple since P_1 learns all the sibling nodes on the path α , and thus it simply follows the accumulation on the sender side. The challenging case is $\omega \neq \alpha$. Let p be the length of the longest common prefix of α and ω (i.e., $\alpha_0, \dots, \alpha_{p-1} = \omega_0, \dots, \omega_{p-1}$) and i be the depth of the sibling nodes of the path ω . The diverging depth p separates these nodes into three categories:

- $0 \leq i < p$: P_1 has the sibling node on level i so it can accumulate the left sibling node to the result as P_0 .
- $i = p$: At the diverging level p , P_1 has $s_p^{(2(\omega|_p) + \omega_p)} = s_p^{(2(\alpha|_p) + \bar{\alpha}_p)}$. If $\omega_p = 0$, the corresponding sibling node $s_p^{(2(\omega|_p) + \bar{\omega}_p)}$ does not contribute to the result so we can safely ignore this case. If $\omega_p = 1$, P_1 must add all the leaf nodes originating from $s_p^{(2(\omega|_p) + \bar{\omega}_p)} = s_p^{(2(\alpha|_p) + \alpha_p)}$ to the result. We explain below how to do this efficiently.
- $i > p + 1$: Since P_1 has the common root of all internal nodes in this case, P_1 works as in the first case.

We now explain the $\omega_p = 1$ case. The α -th leaf node of P_1 is computed as the negative sum of all other leaf nodes, which renders the sum of all leaf nodes to be zero. This implies that

$$\sum_{i \in \{0,1\}^h \text{ s.t. } i|_{p+1} = \alpha|_{p+1}} s_h^{(i)} = - \sum_{i \in \{0,1\}^h \text{ s.t. } i|_{p+1} \neq \alpha|_{p+1}} s_h^{(i)},$$

Parameters: Depth $h \in \mathbb{N}^+$. Field $\mathbb{K}$ with $|\mathbb{K}| \geq 2^\lambda$. Hash function $H : \mathbb{K} \rightarrow \mathbb{K}$.

cGGM.Setup($\Delta \in \mathbb{K}, k \in \mathbb{K}$):

- 1: $(s_0^{(0)}, s_0^{(1)}) := (k, \Delta - k) \in \mathbb{K}^2$
- 2: **for** $i \in [1, h-1], j \in [2^i]$ **do**
- 3: $s_i^{(2j)} := H(s_{i-1}^{(j)}) \in \mathbb{K}$
- 4: $s_i^{(2j+1)} := s_{i-1}^{(j)} - s_i^{(2j)} \in \mathbb{K}$
- 5: **for** $i \in [h]$ **do** $\text{sum}_0^{(i)} := \sum_{j \in [2^i]} s_i^{(2j)} \in \mathbb{K}$
- 6: **return** $\{\text{sum}_0^{(i)}\}_{i \in [h]}$

cGGM.PuncSetup($\alpha \in \{0, 1\}^h, \{\text{sum}_{\alpha_i}^{(i)}\}_{i \in [h]} \in \mathbb{K}^h$):

- 1: $s_0^{(\bar{\alpha}_0)} := \text{sum}_{\alpha_0}^{(0)} \in \mathbb{K}$
- 2: **for** $i \in [1, h-1]$ **do**
- 3: **for** $j \in [2^i], j \neq \alpha|_i$ **do**
- 4: $s_i^{(2j)} := H(s_{i-1}^{(j)}) \in \mathbb{K}$
- 5: $s_i^{(2j+1)} := s_{i-1}^{(j)} - s_i^{(2j)} \in \mathbb{K}$
- 6: $s_i^{(2(\alpha|_i) + \bar{\alpha}_i)} := \text{sum}_{\alpha_i}^{(i)} - \sum_{j \in [2^i], j \neq \alpha|_i} s_i^{(2j + \bar{\alpha}_i)}$
- 7: $s_{h-1}^{(\alpha)} := - \sum_{i \in [h]} s_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \in \mathbb{K}$
- 8: **return** $k_\alpha := (\{s_i^{(2(\alpha|_i) + \bar{\alpha}_i)}\}_{i \in [h]}, s_{h-1}^{(\alpha)})$

cGGM.Acc($\omega \in \{0, 1\}^h, \Delta \in \mathbb{K}, k \in \mathbb{K}$):

- 1: $\text{acc} := 0 \in \mathbb{K}$
- 2: $(s_0^{(0)}, s_0^{(1)}) := (k, \Delta - k) \in \mathbb{K}^2$
- 3: **if** $\omega_0 = 1$ **then** $\text{acc} := \text{acc} + s_0^{(\bar{\omega}_0)} \in \mathbb{K}$
- 4: **for** $i \in [1, h-1]$ **do**
- 5: $s_i^{(2(\omega|_i))} := H(s_{i-1}^{(\omega|_i)}) \in \mathbb{K}$
- 6: $s_i^{(2(\omega|_i)+1)} := s_{i-1}^{(\omega|_i)} - s_i^{(2(\omega|_i))} \in \mathbb{K}$
- 7: **if** $\omega_i = 1$ **then** $\text{acc} := \text{acc} + s_i^{(2(\omega|_i) + \bar{\omega}_i)} \in \mathbb{K}$
- 8: $\text{acc} := \text{acc} + s_{h-1}^{(\omega)} \in \mathbb{K}$
- 9: **return** acc

cGGM.PuncAcc($\omega \in \{0, 1\}^h, \alpha \in \{0, 1\}^h, k_\alpha \in \mathbb{K}^{h+1}$):

- 1: $\text{acc} := 0 \in \mathbb{K}$
- 2: **if** $\omega = \alpha$ **then**
- 3: **for** $i \in [h]$ **do**
- 4: **if** $\omega_i = 1$ **then** $\text{acc} := \text{acc} + s_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \in \mathbb{K}$
- 5: $\text{acc} := \text{acc} + s_{h-1}^{(\alpha)} \in \mathbb{K}$
- 6: **else**
- 7: Let $p \in [h]$ s.t. $\omega|_p = \alpha|_p$ is the longest prefix
- 8: **for** $i \in [p]$ **do**
- 9: **if** $\omega_i = 1$ **then** $\text{acc} := \text{acc} + s_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \in \mathbb{K}$
- 10: **if** $\omega_p = 1$ **then**
- 11: $\text{acc} := \text{acc} - \sum_{i \in [p+1]} s_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \in \mathbb{K}$
- 12: **for** $i \in [p+1, h-1]$ **do**
- 13: $s_i^{(2(\omega|_i))} := H(s_{i-1}^{(\omega|_i)}) \in \mathbb{K}$
- 14: $s_i^{(2(\omega|_i)+1)} := s_{i-1}^{(\omega|_i)} - s_i^{(2(\omega|_i))} \in \mathbb{K}$
- 15: **if** $\omega_i = 1$ **then** $\text{acc} := \text{acc} + s_i^{(2(\omega|_i) + \bar{\omega}_i)} \in \mathbb{K}$
- 16: $\text{acc} := \text{acc} + s_{h-1}^{(\omega)} \in \mathbb{K}$
- 17: **return** acc

Figure 3: Setup and accumulation for cGGM trees.

Functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$

This functionality runs with sender P_0 and receiver P_1 .

Parameters: Field $\mathbb{F}$ and its exponentially large extension $\mathbb{K}$. An upper bound $n \in \text{poly}(\lambda)$.

Initialize: This interface is called only once and ahead of other interfaces. Upon receiving (init) from both parties,

1. If P_0 is corrupted, receive $\Delta \in \mathbb{K}$ from the adversary; otherwise, sample $\Delta \leftarrow \mathbb{K}$. Store Δ , initialize a counter $\text{cnt} := 0$, and output Δ to P_0 .

Extend: Upon receiving (extend) from both parties,

1. If $\text{cnt} + 1 > n$, abort; otherwise, set $\text{cnt} := \text{cnt} + 1$.
2. If P_0 is corrupted, receive $K(x) \in \mathbb{K}$ from the adversary; otherwise, sample $K(x) \leftarrow \mathbb{K}$.
3. If P_1 is corrupted, receive $(x, M(x)) \in \mathbb{F} \times \mathbb{K}$ from the adversary and $K(x) := M(x) - x \cdot \Delta \in \mathbb{K}$; otherwise, sample $x \leftarrow \mathbb{F}$ and $M(x) := K(x) + x \cdot \Delta \in \mathbb{K}$.
4. Output $K(x)$ to P_0 and $(x, M(x))$ to P_1 .

Bootstrap: Upon receiving (bootstrap) from both parties, reset $\text{cnt} := 0$.

Global-key queries: When P_1 is corrupted, upon receiving (guess, Δ') from the adversary, send (success) to the adversary if $\Delta = \Delta'$; send (fail) to the adversary otherwise.

Figure 4: Functionality of streaming PCG for sVOLE.

where the right-hand value equals the negative sum of all sibling nodes on the path α until the $(p+1)$ -th level, as shown in Line 11 of `cGGM.PuncAcc`.

In summary, P_1 computes the leaf-node accumulation from the root level to the leaf level, according to the cGGM invariant and the locally patched α -th leaf node. Before the diverging level, P_1 adds the left sibling nodes in the key k_α to the final result. At the diverging level, if $\omega_p = 1$ then it subtracts all the sibling nodes before (and including) the p -th level from the final result. Finally, it adds all the left sibling nodes from the $(p+1)$ -th level to the leaf level, as well as the ω -leaf node, to the final result. Correctness of this accumulation algorithm is given in Lemma 1, whose proof is deferred to Appendix A.

Lemma 1 (Correctness of cGGM-tree accumulation). *For any $\alpha, \omega \in \{0, 1\}^h$ and $\Delta, k \in \mathbb{K}$, it holds that*

$$\Pr \left[\begin{array}{l} \{\text{sum}_0^{(i)}\}_{i \in [h]} \leftarrow \text{cGGM.Setup}(\Delta, k), \\ \forall i \in [h] : \text{sum}_{\bar{\alpha}_i}^{(i)} := \bar{\alpha}_i \cdot \Delta + (-1)^{\bar{\alpha}_i} \cdot \text{sum}_0^{(i)}, \\ k_\alpha := \text{cGGM.PuncSetup}(\alpha, \{\text{sum}_{\bar{\alpha}_i}^{(i)}\}_{i \in [h]}), : \quad x - y = f_{\alpha, \Delta}^{\geq}(\omega) \\ x := \text{cGGM.Acc}(\omega, \Delta, k), \\ y := \text{cGGM.PuncAcc}(\omega, \alpha, k_\alpha) \end{array} \right] = 1.$$

4.2 Streaming PCG

We model the streaming PCG for sVOLE as an ideal functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ in Fig. 4. It is parameterized by a fixed upper bound n on the number of generated sVOLE tuples and can be bootstrapped to output another at most n sVOLE tuples if this bound has been reached. In this

Protocol $\Pi_{\text{sPCG-sVOLE}}$

Let $N, t \in \mathbb{N}^+$ and $\mathcal{B}$ be a distribution over $\mathbb{F}^N$.

Initialize: The two parties *interactively* proceed as follows:

1. The two parties set $\text{cnt} := 0$ and send (init) to $\mathcal{F}_{\text{sVOLE}}$, which outputs $\Delta \in \mathbb{K}$ to P_0 .
2. Let $h := \log \frac{N}{t} \leq \lambda$. For each $i \in [t]$ in parallel:
 - (a) The two parties send (extend, $h+1$) to $\mathcal{F}_{\text{sVOLE}}$, which outputs $K(s_i) \in \mathbb{K}^{h+1}$ to P_0 and $(s_i, M(s_i)) \in \mathbb{F}^{h+1} \times \mathbb{K}^{h+1}$ to P_1 such that $M(s_i) = K(s_i) + s_i \cdot \Delta$.
 - (b) P_1 samples $r_i \leftarrow \{0, 1\}^h$, defines $\beta_i := s_i^{(0)} \in \mathbb{F}$, $M(\beta_i) := M(s_i^{(0)}) \in \mathbb{K}$ and $M(r_i) := r_i \cdot M(\beta_i) - M(s_i^{(1,h)}) \in \mathbb{K}^h$, and sends $d_i := s_i^{(1,h)} - r_i \cdot \beta_i \in \mathbb{F}^h$ to P_0 .
 P_0 defines $K(\beta_i) := K(s_i^{(0)}) \in \mathbb{K}$ and $K(r_i) := -K(s_i^{(1,h)}) - d_i \cdot \Delta \in \mathbb{K}^h$ such that $M(\beta_i) = K(\beta_i) + \beta_i \cdot \Delta$ and $M(r_i) = K(r_i) + r_i \cdot K(\beta_i)$.
 - (c) P_0 samples $c_i^{(0)} \leftarrow \mathbb{K}$ and computes $k_i := -K(r_i^{(0)}) + c_i^{(0)} \in \mathbb{K}$,
$$\{\text{sum}_{i,0}^{(j)}\}_{j \in [h]} := \text{cGGM.Setup}(K(\beta_i), k_i),$$

$$\forall j \in [1, h-1]: \quad c_i^{(j)} := K(r_i^{(j)}) + \text{sum}_{i,0}^{(j)}.$$
 - (d) P_0 sends $c_i \in \mathbb{K}^h$ to P_1 . P_1 defines $\alpha_i = \alpha_{i,0} \dots \alpha_{i,h-1} := \bar{r}_i^{(0)} \dots \bar{r}_i^{(h-1)} \in \{0, 1\}^h$ and computes
$$\{\text{sum}_{i,\alpha_i,j}^{(j)}\}_{j \in [h]} := ((-1)^{r_i^{(0)}}, \dots, (-1)^{r_i^{(h-1)}}) \odot (-M(r_i) + c_i),$$

$$k_{i,\alpha_i} := \text{cGGM.PuncSetup}(\alpha_i, \{\text{sum}_{i,\alpha_i,j}^{(j)}\}_{j \in [h]}).$$
3. **CONSISTENCY CHECK (ONLY FOR MALICIOUS SECURITY):** Let $\deg(\mathbb{K}/\mathbb{F}) = \mu$ and $X \in \mathbb{K}$. Let $\mathcal{U}(\mathbb{K}, N) = \{U : \mathbb{K}^N \rightarrow \mathbb{K}\}$ denote a set of linear ϵ -universal hash functions, each consisting of N random coefficients $\chi_0, \dots, \chi_{N-1} \in \mathbb{K}$.
 - (a) The two parties send (extend, μ) to $\mathcal{F}_{\text{sVOLE}}$, which outputs $K(m) \in \mathbb{K}^\mu$ to P_0 and $(m, M(m)) \in \mathbb{F}^\mu \times \mathbb{K}^\mu$ to P_1 such that $M(m) = K(m) + m \cdot \Delta$. Moreover, the two parties invoke $\mathcal{F}_{\text{Rand}}$ to sample $U \leftarrow \mathcal{U}(\mathbb{K}, N)$.
 - (b) P_1 defines $e \in \mathbb{F}^N$ such that $e^{(i \cdot \frac{N}{t} + \alpha_i)} = \beta_i$ for each $i \in [t]$ and $e^{(i)} = 0$ otherwise. P_1 parses $u := U(e) \in \mathbb{K}$ as $u \in \mathbb{F}^\mu$ such that $u = \sum_{i \in [\mu]} u^{(i)} \cdot X^i$, and sends $z := m + u \in \mathbb{F}^\mu$ to P_0 .
 - (c) P_0 defines $K(e) \in \mathbb{K}^N$ such that, for each $i \in [t]$ and $j \in [\frac{N}{t}]$, $K(e^{(i \cdot \frac{N}{t} + j)})$ equals the j -th leaf node of its i -th cGGM tree. P_0 computes $v := U(K(e)) + \sum_{i \in [\mu]} (K(m^{(i)}) + z^{(i)} \cdot \Delta) \cdot X^i \in \mathbb{K}$.
 - (d) P_1 defines $M(e) \in \mathbb{K}^N$ such that, for each $i \in [t]$ and $j \in [\frac{N}{t}]$, $M(e^{(i \cdot \frac{N}{t} + j)})$ equals the j -th leaf node of its i -th *punctured* cGGM tree (whose nodes are defined as per cGGM.PuncSetup) plus $\mathbb{1}(j = \alpha_i) \cdot M(\beta_i)$. P_1 computes $w := U(M(e)) + \sum_{i \in [\mu]} M(m^{(i)}) \cdot X^i \in \mathbb{K}$. The two parties invoke $\mathcal{F}_{\text{EQ}}$ to check whether $w = v$ or not. If either party receives 0 from $\mathcal{F}_{\text{EQ}}$, it aborts; otherwise, it continues.
4. P_0 stores $(\Delta, \{k_i, K(\beta_i)\}_{i \in [t]})$ and P_1 stores $\{\alpha_i, \beta_i, k_{i,\alpha_i}, M(\beta_i)\}_{i \in [t]}$. P_0 outputs Δ .

Figure 5: Protocol of streaming PCG for sVOLE (with the relaxed regular noise e), Part I.

streaming PCG, a corrupted receiver is allowed to query the sVOLE global key Δ of the sender. This global-key query does not reduce the effective security and is considered by many PCG works (e.g., [YW⁺20, WYKW21, GYW⁺23]).

Our streaming PCG protocol $\Pi_{\text{sPCG-sVOLE}}$ for the sVOLE correlation is presented in Fig. 5.

Protocol $\Pi_{\text{sPCG-sVOLE}}$ (continued)

Extend: The two parties *locally* proceed as follows:

1. If $\text{cnt} + 1 > n$, the two parties abort; otherwise, they set $\text{cnt} := \text{cnt} + 1$.
2. The two parties sample $\mathbf{b} \leftarrow \mathcal{B}$ using the same randomness. Assume that $\mathbf{b}$ has $\ell \in \mathbb{N}^+$ non-zero entries in $\mathbb{F}$ only at the indices in $\mathcal{J}$ such that for each $j \in \mathcal{J}$, $j = \mu_j \cdot \frac{N}{t} + \omega_j \in [N]$, where $\mu_j \in [t]$ and $\omega_j \in [\frac{N}{t}]$.
3. P_0 outputs

$$\mathbf{K}(x) := \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \begin{cases} 0, & \mu_j < i \\ \text{cGGM.Acc}(\omega_j, \mathbf{K}(\beta_i), k_i), & \mu_j = i \\ \mathbf{K}(\beta_i), & \mu_j > i \end{cases}$$

4. P_1 outputs $x := \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \mathbb{1}(j \geq i \cdot \frac{N}{t} + \alpha_i) \cdot \beta_i$ and

$$\mathbf{M}(x) := \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \begin{cases} 0, & \mu_j < i \\ \mathbb{1}(\omega_j \geq \alpha_i) \cdot \mathbf{M}(\beta_i) + \text{cGGM.PuncAcc}(\omega_j, \alpha_i, k_{i, \alpha_i}), & \mu_j = i \\ \mathbf{M}(\beta_i), & \mu_j > i \end{cases}$$

Bootstrap: The two parties reset $\text{cnt} := 0$ and *interactively* rerun Item 2 to Item 4 of **Initialize** to produce fresh $\{k_i, \mathbf{K}(\beta_i)\}_{i \in [t]}$ for P_0 and $\{\alpha_i, \beta_i, k_{i, \alpha_i}, \mathbf{M}(\beta_i)\}_{i \in [t]}$ for P_1 .

Figure 5: Protocol of streaming PCG for sVOLE (with the relaxed regular noise $\mathbf{e}$), Part II.

This protocol relies on our cGGM-tree accumulation algorithms (Fig. 3), which makes the correlation generation sublinear-time without using the heavy relaxed distributed comparison function [BCG⁺22]. In this protocol, we can implement the **Bootstrap** interface by using $t(\log \frac{N}{t} + 1) + \mu$ sVOLE tuples generated by the protocol itself in **Extend**, instead of calling a subroutine $\mathcal{F}_{\text{sVOLE}}$ functionality, which is called only once in the **Initialize** phase and can be implemented with non-PCG-style extension protocols (e.g., [IKNP03, KOS15, Roy22, DPSZ12, NNOB12, KOS16]).

Jumping ahead, based on our countermeasure to resist our EA-LPN attacks (see Section 5.2), the noise $\mathbf{e}$ in our protocol is sampled such that one regular index is uniformly chosen for every $\frac{N}{t}$ -size block and its associated payload is uniform over $\mathbb{F}$ instead of $\mathbb{F}^*$ as in previous work [BCG⁺22]. To support such a noise, we can use a random sVOLE tuple $(\mathbf{K}(\beta), \beta, \mathbf{M}(\beta))$ distributed between the two parties, where $\beta \leftarrow \mathbb{F}$, without sending a difference to correct it to a non-zero value. When $\mathbb{F} = \mathbb{F}_2$, this is also different from the cGGM-based COT protocol [GYW⁺23] where the cGGM-tree offset is exactly the COT global key Δ to make a non-zero $\beta = 1$. Compared to this COT protocol, although ours has one more round to send the differences $\{\mathbf{d}_i\}$, we can have better EA-LPN parameters to minimize the overall computation by reducing the Hamming weight ℓ of each vector $\mathbf{b}$.

In the **Initialize** phase, we process each noise block in parallel as follows. For each block, we need to generate the single-point noise as well as the corresponding authentication information, which can be done by the sender setting the cGGM-tree offset to $\mathbf{K}(\beta)$ and the receiver adding a correction term $\mathbf{M}(\beta)$ to the patched α -th leaf node. This cGGM tree is prepared to run the accumulation algorithms in the subsequent phase. During this phase, the sender may send incorrect correction messages $\{\mathbf{c}_i\}$. To handle this, we modify the consistency check in the Ferret protocol [YWL⁺20], which is designed to check a GGM tree, to the cGGM tree. This check and the

EA-LPN assumption with static leakage jointly guarantee the security against a malicious sender. When $\mathbb{F} \neq \mathbb{F}_2$, the consistency check also prevents the malicious receiver to use a non-binary $\mathbf{r}_i \in \mathbb{F}^h$ as its chosen path by aborting the sender.

In the **Extend** phase, the two parties use their common randomness to sample a row $\mathbf{b}$ of the expander matrix $\mathbf{B}$ in the EA-LPN assumption. This can be done on-the-fly. Since we set the EA-LPN parameters for a bounded number n of sVOLE tuples, we introduce a counter to keep track of how many correlations have been generated. Once the counter reaches n , indicating that noise vector $\mathbf{e}$ has been fully saturated, we can run the setup algorithm once again to generate a fresh noise and the corresponding local keys.

Security. The correctness of this protocol follows from the correctness of the cGGM accumulation (Lemma 1). The security against the malicious and semi-honest adversary is proved in Theorem 1 and Theorem 2, where the semi-honest security no longer requires the EA-LPN assumption with static leakage as the consistency check is removed. We postpone their proofs to Appendix C and Appendix D. In these proofs, the linear-orthomorphism-based hash construction follows from that required by a cGGM tree [GYW⁺23].

Theorem 1. *In the $(\mathcal{F}_{\text{sVOLE}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{EQ}})$ -hybrid model and under the EA-LPN-SL($t, n, N, \ell, \mathbb{F}$) assumption, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any malicious adversary, if hash function $\mathbf{H}(x) = \pi(\sigma(x)) + \sigma(x)$, where $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a (programmable) random permutation and $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an efficient linear orthomorphism².*

Theorem 2. *In the $\mathcal{F}_{\text{sVOLE}}$ -hybrid model and under the EA-LPN($t, n, N, \ell, \mathbb{F}$) assumption, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5, but removing the consistency check) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any semi-honest adversary, if hash function $\mathbf{H}(x) = \pi(\sigma(x)) + \sigma(x)$, where $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a (programmable) random permutation and $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an efficient linear orthomorphism.*

Complexity analysis. $\Pi_{\text{sPCG-sVOLE}}$ requires t cGGM trees. The black-box distributed setup of the cGGM trees and the consistency check consumes $t(\log \frac{N}{t} + 1) + \mu$ random pre-computed sVOLE tuples in total. In this setup, the receiver sends $(\{\mathbf{d}_i\}_i, \mathbf{z})$ of $(t \log \frac{N}{t} + \mu) \log |\mathbb{F}|$ bits; the sender sends $\{\mathbf{c}_i\}_i$ of $t \log \frac{N}{t} \log |\mathbb{K}|$ bits.

The computation of the sender is dominated by the two on-the-fly expansion (due to the sub-linear memory) of its cGGM tree in Item 2c and Item 3c, incurring $2N$ block-cipher calls, and a universal hashing of size N . The same computation holds for the receiver.

The extension phase is non-interactive and the computation is dominated by the evaluation of at most ℓ cGGM paths, causing overall $\ell \log \frac{N}{t}$ block-cipher calls. The bootstrapping phase have the same computation and communication as the distributed setup phase, except that the pre-computed sVOLEs are from the previous $t(\log \frac{N}{t} + 1) + \mu$ extensions among the allowed n extensions.

Optimization when $\mathbb{F} \neq \mathbb{F}_2$. As we noted in Section 5.1, our attacks offer limited advantage in the case of a large field $\mathbb{F}$. Therefore, one can still use the regular noise in our streaming PCG. To achieve this, the receiver can locally sample a non-zero $\beta_i \leftarrow \mathbb{F}^*$ and sends a difference $\delta_i := \mathbf{s}_i^{(0)} - \beta_i$

²A mapping $\sigma : \mathbb{G} \rightarrow \mathbb{G}$ for an additive Abelian group $\mathbb{G}$ is a linear orthomorphism if (i) σ is a permutation, (ii) $\sigma(x + y) = \sigma(x) + \sigma(y)$ for any $x, y \in \mathbb{G}$, and (iii) $\sigma'(x) := \sigma(x) - x$ is also a permutation. [GKWWY20] presents two efficient instantiations of σ (with well-defined efficient σ^{-1} , σ' , and σ'^{-1}): (i) if $\mathbb{G}$ is a field, $\sigma(x) := c \cdot x$ for some $c \neq 0, 1 \in \mathbb{G}$, and (ii) if $\mathbb{G} = \{0, 1\}^n$, $\sigma(x) = \sigma(x_L \parallel x_R) := (x_L \oplus x_R) \parallel x_L$ where x_L and x_R are the left and right halves of x .

Functionality $\mathcal{F}_{\text{PCF-sVOLE}}$

This functionality runs with sender P_0 and receiver P_1 .

Parameters: Field $\mathbb{F}$ and its exponentially large extension $\mathbb{K}$.

Initialize: This interface is called only once and ahead of other interfaces. Upon receiving (init) from both parties,

1. If P_0 is corrupted, receive $\Delta \in \mathbb{K}$ from the adversary; otherwise, sample $\Delta \leftarrow \mathbb{K}$. Store Δ and output Δ to P_0 .

Extend: This interface can be called for any polynomially many times. Upon receiving (extend) from both parties,

1. If P_0 is corrupted, receive $K(x) \in \mathbb{K}$ from the adversary; otherwise, sample $K(x) \leftarrow \mathbb{K}$.
2. If P_1 is corrupted, receive $(x, M(x)) \in \mathbb{F} \times \mathbb{K}$ from the adversary and $K(x) := M(x) - x \cdot \Delta \in \mathbb{K}$; otherwise, sample $x \leftarrow \mathbb{F}$ and $M(x) := K(x) + x \cdot \Delta \in \mathbb{K}$.
3. Output $K(x)$ to P_0 and $(x, M(x))$ to P_1 .

Global-key queries: When P_1 is corrupted, upon receiving (guess, Δ') from the adversary, send (success) to the adversary if $\Delta = \Delta'$; send (fail) to the adversary otherwise.

Figure 6: Functionality of PCF for sVOLE.

to the sender to turn the two-party IT-MAC for $s_i^{(0)}$ into that for $\beta_i \neq 0$, instead of using a possibly zero $s_i^{(0)}$ as β_i directly. This change requires only a little additional communication but, by using the regular noise instead of the relaxed regular noise when $\mathbb{F} \neq \mathbb{F}_2$, we can avoid a doubled t (see Section 5.2), maintaining the same security and saving the memory, the overall communication, and the number of pre-computed tuples.

Optimization when $\mathbb{F} = \mathbb{F}_2$. In this case, the malicious receiver can only choose a binary $r_i \in \mathbb{F}^h$ so that the consistency check can be simplified to the one-side check of [YWL⁺20], where the receiver samples the linear universal hash and sends it to the sender. This allows the receiver to compute the hash of all its leaf nodes incrementally during the tree expansion, instead of recomputing these nodes in the consistency check. As a result, the computation of the receiver can be optimized to N block-cipher calls.

4.3 Extension to PCF

We stress that our cGGM-tree accumulation algorithms can also be applied to construct a PCF for sVOLE correlations since their computation complexity is logarithmic in the super-polynomial bound n of generated correlations in PCF. Similar to our streaming PCG, we model this PCF as an ideal functionality $\mathcal{F}_{\text{PCF-sVOLE}}$, which is presented in Fig. 6. In this functionality, the two parties can request an arbitrarily but polynomially many sVOLE correlations. By using malicious generic (e.g., secret-sharing-based) 2PC (e.g., [NNOB12, DPSZ12]) to run the non-black-box setup to compute the on-path and adjacent cGGM nodes of a path α (instead of the black-box one in Item 2d of protocol $\Pi_{\text{sPCG-sVOLE}}$), we have the following theorem.

Theorem 3. *Under the EA-LPN($t, n, N, \ell, \mathbb{F}$) assumption, there exists a protocol $\Pi_{\text{PCF-sVOLE}}$ UC-realizes the functionality $\mathcal{F}_{\text{PCF-sVOLE}}$ (Fig. 6) against any malicious adversary. Its computation and*

communication is $O(C)$ for the circuit complexity $C = t(\log \frac{N}{t} - 1) \cdot |\mathbf{H}|$ of the cGGM setup, where $|\mathbf{H}|$ is the circuit complexity of hash function $\mathbf{H}$.

Note that the functionality will inherit the global-key queries of the corrupted receiver due to the underlying cGGM invariant. As the malicious follows from the malicious 2PC, we use the EA-LPN assumption, without considering the static leakage in the consistency check.

5 Cryptanalysis against EA-LPN

Boyle et al. [BCG⁺22] proposed EA-LPN parameters by setting $N = 5n$ and choosing $\ell \in \{7, 9, 11\}$. The value of t is chosen from an empirical estimate of the average minimum row weight of $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell)$, modeled by an exponential regression in $\log(n)$. More specifically, the minimum distance of $\mathbf{H}$ is approximately δN , where δ depends on ℓ and n as follows:

$$\delta \approx \begin{cases} 0.4646 \cdot e^{-0.1012 \cdot \log(n)}, & \text{if } \ell = 7 \\ 0.4415 \cdot e^{-0.0783 \cdot \log(n)}, & \text{if } \ell = 9 \\ 0.4337 \cdot e^{-0.06383 \cdot \log(n)}, & \text{if } \ell = 11. \end{cases} \quad (5)$$

Finally, t is chosen to satisfy $t \geq \frac{128 - \log(N)}{2 \log(e) \cdot \delta}$.

5.1 Our Attacks against EA-LPN

We propose two attacks: a recovery attack and a distinguishing attack. These attacks exploit the structure of $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell, \mathcal{R})$, where each row consists of $\ell + 1$ consecutive equal-value blocks. The inner product between such a block and a regular noise block (with Hamming weight 1) reduces to multiplying a known value with an unknown non-zero variable. Over $\mathbb{F}_2$, the only non-zero value is 1, so the inner product between the equal-value block (with value a) and the regular noise (with Hamming weight t) simplifies to $a \cdot t \bmod 2$ for public a , resulting in a deterministic outcome without the need for guessing. However, in larger fields, the noise can take on many non-zero values, so the unknown variable has many possible values, introducing sufficient randomness to resist our attacks. As a result, our attacks are highly effective in the case of $\mathbb{F}_2$, but offer limited advantage in the case of larger fields.

Recovery attack. This attack reduces the EA-LPN problem to a lower-dimensional primal-LPN instance by introducing a new noise Δ , using a tunable parameter $\alpha \in \mathbb{N}$ to control the trade-off and allow efficient recovery. Specifically, it transforms the EA-LPN instance

$$(\mathbf{H} = \mathbf{B} \cdot \mathbf{A}, \mathbf{b} = \mathbf{H} \cdot \mathbf{e})$$

to a primal-LPN instance $(\widehat{\mathbf{H}}, \mathbf{b}' = \widehat{\mathbf{H}} \cdot \mathbf{x} + \Delta)$, where $\widehat{\mathbf{H}}$ has low dimension and Δ has low Hamming weight. This attack is as follows, assuming $N = 2\alpha\beta$ with $\beta \in \mathbb{N}$.

1. Construct a modified matrix $\mathbf{H}'$ and a low-weight noise vector Δ such that $\mathbf{b}' = \mathbf{H}' \cdot \mathbf{e} + \Delta$. Concretely, each row of $\mathbf{H}'$ is formed by replacing consecutive blocks (each of length 2α) with the majority value of the corresponding blocks in $\mathbf{H}$. More specifically, for each $i \in [n]$ and $j \in [0, \beta - 1]$, if there exist $\gamma \in [\lfloor \frac{2\alpha t}{N} \rfloor]$ and $\eta \in [\lfloor \frac{N}{t} \rfloor]$ such that $\mathbf{B}^{(i, 2\alpha j + \gamma \cdot \lfloor \frac{N}{t} \rfloor + \eta)} \neq 0$, then for $k \in [2\alpha]$, define

$$\mathbf{H}'^{(i, 2\alpha j + k)} := \text{Maj} \left(\left\{ \mathbf{H}^{(i, 2\alpha j + \gamma' \cdot \lfloor \frac{N}{t} \rfloor + \eta')} \mid \forall \eta' \in [\lfloor \frac{N}{t} \rfloor] \right\} \right);$$

otherwise, $\mathbf{H}'^{(i, 2\alpha j + k)} := 0$. Define the modified output vector $\mathbf{b}'$ by flipping each bit of $\mathbf{b}$ based on the majority value within each noise block of $\Delta \mathbf{H} := \mathbf{H}' - \mathbf{H}$:

$$\mathbf{b}'^{(i)} = \mathbf{b}^{(i)} + \sum_{j=0}^{t-1} \text{Maj} \left(\left\{ \Delta \mathbf{H}^{(i, j \cdot \lfloor \frac{N}{t} \rfloor + k)} \mid \forall k \in [\lfloor \frac{N}{t} \rfloor] \right\} \right).$$

Here, the Hamming weight of $\Delta = \mathbf{b}' - \mathbf{H}' \cdot \mathbf{e}$ is small, since both $\mathbf{B}$ and $\mathbf{e}$ have small Hamming weights.

2. Compress the matrix $\mathbf{H}'$ to obtain $\widehat{\mathbf{H}} \in \mathbb{F}^{n \times \beta}$ by selecting columns at fixed intervals:

$$\widehat{\mathbf{H}}^{(\cdot, j)} = \mathbf{H}'^{(\cdot, 2\alpha j)}, \text{ for each } j \in [\beta].$$

Similarly, compress $\mathbf{e}$ into $\mathbf{x}$ as follows:

$$\mathbf{x}^{(k)} = \sum_{i=(2k-2) \cdot \alpha + 1}^{2k \cdot \alpha} \mathbf{e}^{(i)}, \text{ for each } k \in [\beta].$$

We choose α to minimize the attack time, i.e., $T = \min_{\alpha \geq 1} T_\alpha$, where T_α is the time needed to distinguish the primal-LPN instance $(\widehat{\mathbf{H}}, \mathbf{b}' = \widehat{\mathbf{H}} \cdot \mathbf{x} + \Delta)$ from a random one.

We analyze the security of the EA-LPN assumption over $\mathbb{F}_2$ with regular noise under the first parameter setting, where $\ell \in \{7, 9, 11\}$ and the noise has a high weight t . For simplicity, we assume that ℓ divides t . By setting $\alpha = \frac{N}{2\ell}$ and $\beta = \frac{N}{2\alpha} = \ell$, we reduce the EA-LPN instance to a primal-LPN problem with sample size n , a regular secret $\mathbf{x}$, and sparse noise Δ .

We focus on the Hamming weight of sparse noise Δ . Recall that the noise vector $\mathbf{e}$ is regular and consists of t blocks, where each block $\mathbf{e}_j \in \mathbb{F}_2^{\lfloor \frac{N}{t} \rfloor}$ with Hamming weight 1. So, we only need to analyze each block independently:

$$\Delta_{ij} := \sum_{k=1}^{\lfloor \frac{N}{t} \rfloor} \left((\Delta \mathbf{H}^{(i, j \cdot \lfloor \frac{N}{t} \rfloor + k)}) \cdot \mathbf{e}_j^{(k)} \right).$$

If the Hamming weight of $\left\{ \mathbf{B}^{(i, j \cdot \lfloor \frac{N}{t} \rfloor + k)} \mid k \in [\lfloor \frac{N}{t} \rfloor] \right\}$ is non-zero, then Δ_{ij} independently follows a distribution close to Bernoulli distribution $\text{Ber}_{1/4}$; otherwise, its value is known. Although $\text{HW}(\mathbf{e}) = t$, each noise term $\Delta^{(i)}$ (for $i \in [n]$) depends on at most ℓ noise blocks. By the piling-up lemma, the noise Δ follows a Bernoulli distribution Ber_η^n with bias $\eta \leq \frac{1}{2} - 2^{-\ell-1}$ (for $\ell \in \{7, 9, 11\}$), which is enough to recover $\mathbf{e}$ from the EA-LPN instance.

Our algorithm exhaustively searches for $\mathbf{x}$ such that the Hamming weight of $\mathbf{b} - \widehat{\mathbf{H}} \cdot \mathbf{x}$ is at most $\frac{n}{2} - \frac{n \cdot 2^{-\ell}}{4}$. Thus, the time complexity of our attack is $T \leq 2^\ell \cdot n \cdot 2\ell$, where 2^ℓ accounts for the exhaustive search over $\mathbf{x}$, and $n \cdot 2\ell$ is the cost of computing $\widehat{\mathbf{H}} \cdot \mathbf{x}$.

Distinguishing attack. This attack builds on the same core idea as our recovery attack. It can be somewhat seen as a special case of our recovery attack with $\beta = 1$, specifically tailored for distinguishing. For EA-LPN over $\mathbb{F}_2$ with regular noise, where $(\mathbf{H} = \mathbf{B} \cdot \mathbf{A}, \mathbf{b} = \mathbf{H} \cdot \mathbf{e})$, we observe that each component of $\mathbf{b}$ can be predicted with probability approximately $1/2 + (1/2)^\ell$, and the bit security is approximately 2ℓ , where ℓ is the Hamming weight of the corresponding row in $\mathbf{B}$. Structurally, each row of $\mathbf{H} \leftarrow \text{EAGen}(n, N, \ell)$ consists of $\ell + 1$ consecutive blocks of all 0s or all 1s, while the regular noise vector $\mathbf{e}$ consists of t equal-length blocks, each containing exactly one

component equal to 1 and all others equal to 0. Therefore, at most ℓ entries in $\mathbf{e}$ influence each component of $\mathbf{b}$. Given an instance $(\mathbf{H}, \mathbf{b})$, either random or generated from an EA-LPN instance, we have a distinguishing attack as follows.

1. We define a modified output vector $\mathbf{b}'$ by flipping each bit of $\mathbf{b}$ based on the majority value within each noise block of $\mathbf{H}$:

$$\mathbf{b}'^{(i)} = \mathbf{b}^{(i)} + \sum_{j=0}^{t-1} \text{Maj}\left(\left\{\mathbf{H}^{(i,j \cdot \lfloor \frac{N}{t} \rfloor + k)} \mid \forall k \in \left[\lfloor \frac{N}{t} \rfloor\right]\right\}\right).$$

2. If $\text{Maj}(\mathbf{b}') = 0$, then $(\mathbf{H}, \mathbf{b})$ is from an EA-LPN instance; otherwise, it is random.

5.2 Our Countermeasure

A modified noise distribution. To defend our attacks, we introduce the relaxed regular noise: in each block, a single position is chosen randomly and assigned a uniform element (possibly zero), while all other entries are zero. This ensures that each entry of $\mathbf{b} = \mathbf{H} \cdot \mathbf{e}$ is influenced by many noise blocks, not just ℓ of them. As a result, the inner product between a row block of $\mathbf{H}$ (filled with a fixed value) and a noise block is sufficiently random. This randomness is further amplified by the large minimum distance of the EA code, making our attack ineffective.

To resist other attacks, such as algebraic attacks [BØ23] and ISD attacks [MMT11, BJMM12, LWYY24, ES24, KL24], we increase the number t of noise blocks so that the total Hamming weight matches in expectation that of regular noise in [BCG⁺22]. Even though each block still contains only one uniform element, this adjustment increases the overall randomness and prevents attackers from exploiting noise patterns.

- Algebraic attacks [BØ23] solve an LPN assumption by converting the regular structure of noise $\mathbf{e}$ into a system of polynomial equations. However, with the relaxed regular noise, each noise block consists of a uniformly random bit in a random position. As the Hamming weight is preserved in expectation, the new noise does not increase the number of equations or reduce the number of variables, and thus does not lower the overall cost of the attack.
- ISD attacks [MMT11, BJMM12, LWYY24, ES24, KL24] attempt to recover the secret by guessing a subset of positions (an “information set”) where the noise is sparse. These attacks are particularly effective when the noise has low Hamming weight. In our setting, the relaxed regular noise maintains a high Hamming weight in expectation, significantly reducing the probability of choosing a subset with sparse noise and thus reducing the cost of ISD attacks.

Parameter selection for EA-LPN with the relaxed regular noise. For the EA-LPN assumption with the relaxed regular noise, we can adopt the parameter selection strategy similar to that of [BCG⁺22], relying on the linear test framework for concrete security estimates. Only algebraic attacks [BØ23] fall outside this framework. However, these attacks are ineffective for EA-LPN with parameters such as $N = 5n$, as they require significantly more samples and mainly apply to the primal form of LPN.

In the linear test framework, the relaxed regular noise offers the same bit security as Bernoulli noise because the attack depends only on the bias of the inner product $\langle \mathbf{a}, \mathbf{b} \rangle$, where $\mathbf{b} = \mathbf{H} \cdot \mathbf{e}$. Given a public matrix $\mathbf{H}$, this bias is determined by the distribution of the noise vector $\mathbf{e}$. If both noise distributions have the same average Hamming weight, they also have the same number of

	N	t	$\log \frac{N}{t}$	ℓ	n
LPN1	9,781,248	1,194	13	7	1,956,249
LPN2	51,478,528	1,571	15	7	10,295,705
LPN3	293,601,280	1,120	18	9	58,720,256

Table 2: Concrete LPN parameter sets for the EA code with the relaxed regular noise, achieving 128-bit security.

		1 Thread				4 Threads				8 Threads				16 Threads			
		Base Comm. (MB)	S. Comm. (MB)	Init. (ms)	Extend (μ s / COT)	Mem. (MB)	Init. (ms)	Extend (μ s / COT)	Mem. (MB)	Init. (ms)	Extend (μ s / COT)	Mem. (MB)	Init. (ms)	Extend (μ s / COT)	Mem. (MB)	Init. (ms)	Extend (μ s / COT)
LPN1	S	0.283	0.251	52.2	0.348	11.9	27.03	0.112	14.8	24.7	0.082	19.1	25.4	0.07	27.0		
	M	0.283	0.257	159	0.356	11.9	57	0.118	14.8	41	0.079	19.3	52	0.068	27.3		
LPN2	S	0.419	0.383	199	0.393	13.2	67.4	0.148	16	59.7	0.09	20	61.6	0.076	27.9		
	M	0.419	0.389	560	0.395	13.1	174	0.165	16.2	134	0.092	20	117	0.082	28.3		
LPN3	S	0.357	0.325	992	0.580	12.1	289	0.21	15.2	293	0.132	19.1	308	0.112	27.3		
	M	0.357	0.33	2925	0.582	12.1	806	0.216	15.2	777	0.140	19.3	741	0.114	27.5		

Table 3: Performance of our streaming PCG for COT under different EA-LPN parameters. Communication is the total for both parties. Memory and computation denote the maximum used by either party. “Base Comm.” is the communication for base COTs while “S. Comm.” refers to only the communication for silent COT extension. “S” denotes semi-honest security while “M” denotes malicious security.

uniform elements. As a result, the resulting bias—and the bit security—remains unchanged. Thus, all linear-test-based security estimates for Bernoulli noise apply directly to the relaxed regular noise.

Following the analysis in [BØ23], we can choose the EA-LPN parameters with the relaxed regular noise by setting $N = 5n$ and $\ell \in \{7, 9, 11\}$. Then, the number of blocks t in our setting is given by $2 \cdot \frac{128 - \log(N)}{2 \log(e) \cdot \delta}$ for the factor δ estimated in (5). Such a t is twice as large as that in [BCG+22].

6 Performance Evaluation

6.1 Experimental Setup

We implement our streaming PCG protocol (in Fig. 5) for the COT case where $\mathbb{F} = \mathbb{F}_2$ and $\mathbb{K} = \mathbb{F}_{2^\lambda}$. This case is of particular interest to achieve silent OT extension [BCG+19b, BCG+19a]. We set the security parameter $\lambda = 128$. Our implementation is written in C++ and uses EMP toolkit [WMK16]. The code can be found at <https://github.com/AntCPLab/OpenDory>.

The evaluation is conducted on a server equipped with Intel Xeon Platinum 8369B CPU (2.70GHz) with 16 cores and 32 GB memory. We consider the LAN setting, where the sender and receiver are on the same physical machine without any extra latency or bandwidth limitation. We also simulate different bandwidth varying from 5 Mbps to 1 Gbps using the `tc` command.

For our streaming PCG, we propose three concrete parameter sets for the EA-LPN assumption according to our countermeasure in Section 5.2, and refer to them as LPN1, LPN2, and LPN3 respectively. The detailed parameters are shown in Table 2, where we have $N = 5n$.

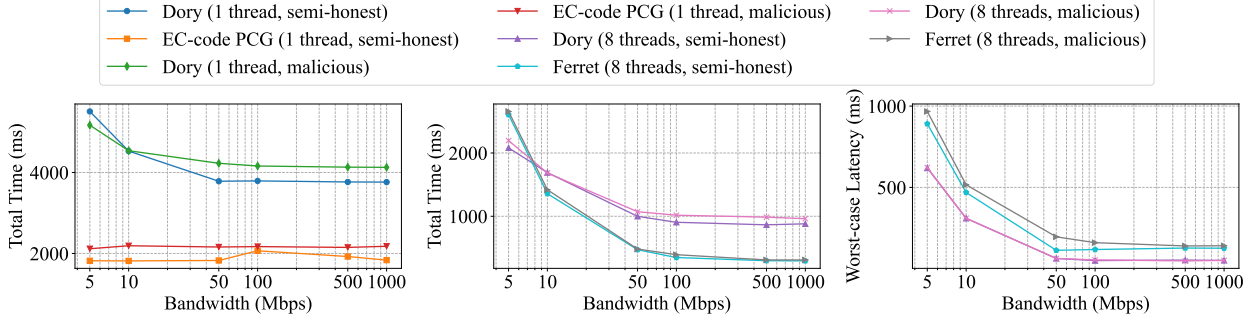


Figure 7: Comparison with the state-of-the-art PCGs when generating 10 million COT tuples. “Total Time” equals the time to generate base COTs plus the time of silent COT extension. “Worst-case Latency” refers to the maximum time (if the randomness of PCG seeds is exhausted) to get the next COT tuple, equaling, based on pre-computed COT tuples, the time of silent COT extension in Ferret [YWL⁺20] and the EC-code PCG [RRT23] or the bootstrapping time of our streaming PCG.

6.2 Benchmark of Our Streaming PCG

We measure the performance of our streaming PCG to generate COTs, and report the initialization time, per-COT extension time, memory usage and the total communication cost. We change the underlying EA-LPN parameters, the number of threads (from 1 to 16), and the security setting (from semi-honest security to malicious security). The results are shown in Table 3.

From these results, we observe that our streaming PCG demonstrates great scalability with multi-threading in the COT case. Taking LPN1 in the semi-honest security for an example, compared to the single thread setting, the per-COT extension time has a $3.1\times$ and $4.4\times$ speedup with 4 and 8 threads, respectively. Yet using 16 threads has only a limited improvement in the time. A similar scaling pattern holds for the other two parameter sets and security setting. This efficient parallelization enables high-throughput COT generation suitable for large-scale applications.

We observe that the memory usage and communication is almost independent of the security setting. Moreover, the memory usage is relatively stable across different parameter sets and thread counts. The two metrics are very low. For example, in LPN3 for generating at most 58,720,256 COTs on-the-fly, running our streaming PCG with 16 threads and malicious security requires only at most 27.5 MB memory per party and 0.687 MB (both-side) communication.

Compared with semi-honest security, the extra overhead to achieve malicious security is merely incurred during the initialization since, given a maliciously secure initialization, the malicious security of the extension is straightforward since it is non-interactive. This extra overhead roughly aligns with our complexity analysis in Section 4.2. We leave it to future work how to avoid the second tree expansion to achieve both malicious security and sublinear memory.

6.3 Comparison with State-of-the-Art PCF

Given that no implementation of [BCG⁺22] is publicly available, we choose to estimate its concrete performance. We focus on the communication of black-box distributed setup when n is polynomial. This communication, without that of base COTs, will be 0.993 MB, 1.508 MB, and 1.29 MB for LPN1, LPN2, and LPN3 setting, respectively. Plus the communication for the same number of base COTs in Table 3, we have an improvement on communication by a factor $2.39\times \sim 2.41\times$, below the expected $4\times$ due to the communication for base COTs. However, this decrease can only happen

for the first-time initialization. The bootstrapping phase can still have the expected improvement.

6.4 Comparison with State-of-the-Art PCGs

Ferret [YWL⁺20] and the EC-code PCG [RRT23] are the two state-of-the-art PCGs for generating random COTs, using primal-LPN and dual-LPN assumption respectively. We use EMP toolkit [WMK16] for the open-source implementation of Ferret and libOTe [PR] for the EC-code PCG.

For the comparison with these PCGs, we measure the total communication, memory usage, end-to-end execution time and worst-case latency in the generation of 10 million COTs under both semi-honest and malicious security. The worst-case latency refers to the maximum delay to produce the next COT tuple and occurs if we run out of the randomness of PCG seeds so that they need to be regenerated. In Ferret and the EC-code PCG, this latency reflects the time of running the full silent OT extension on some pre-computed COT tuples. In our streaming PCG, it is exactly the bootstrapping time. Unfortunately, libOTe does not provide multi-threading and a network interface to measure the worst-case latency. Therefore, our comparison to the EC-code PCG considers single thread and does not measure the worst-case latency.

To generate 10 million COT tuples, our streaming PCG uses the parameter set LPN2; Ferret and the EC-code PCG use the parameters selected in their implementations. The parameters ensure that their workloads need no bootstrapping. We benchmark the performance with different bandwidth. The results are shown in Fig. 7 and as follows.

The memory cost and communication of our streaming PCG are 20 MB and 0.8 MB, respectively, while the two metrics are 337 MB and 1.5 MB for Ferret, and 624 MB and 0.19 MB for the EC-code PCG. In terms of memory, we achieve $16.7\times$ and $31.2\times$ improvements on Ferret and the EC-code PCG. Our communication is 50% of that sent in Ferret but higher than the EC-code PCG since EC code enables concretely smaller N and t . Using a significantly smaller memory and being able to generate correlations on-the-fly, our streaming PCG is slower than the two PCGs for its computation growing sublinearly in N . However, the resulting computation is still reasonable. When it comes to the worst-case latency, our streaming PCG achieves a $2\times$ improvement over Ferret due to the cGGM-tree structure.

We can regard the low memory and high computation in our streaming PCG as a trade-off, leading to a memory-efficient architecture. This characteristic becomes critically advantageous in large-scale applications demanding high parallelism: Ferret and the EC-code PCG exhibit a significant memory scalability challenge when instantiating a substantial number of COTs required for massively parallel workflows. In such scenarios, our streaming COT generation and low-memory design render it a pragmatically superior solution for memory-constrained environments. Moreover, the small memory footprint means that the entire workload can fit into the L3 cache, thus accelerating computation.

Acknowledgements

The work of Yu Yu is supported by the National Key Research and Development Program of China (Grant No. 2020YFA0309705), National Natural Science Foundation of China (Grant Nos. 62125204 and 92270201) and Innovation Program for Quantum Science and Technology (No. 2021ZD0302901 / 2021ZD0302902). The work of Xiao Wang and Hanlin Liu is supported by NSF award #2236819. The work of Yu Yu is also supported by Ant Group.

References

- [ABL⁺18] David W. Archer, Dan Bogdanov, Yehuda Lindell, Liina Kamm, Kurt Nielsen, Jakob Illeborg Pagter, Nigel P. Smart, and Rebecca N. Wright. From keys to databases - real-world applications of secure multi-party computation. *Comput. J.*, 61(12):1749–1771, 2018.
- [BCG⁺19a] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 291–308. ACM Press, November 2019.
- [BCG⁺19b] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- [BCG⁺20a] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Correlated pseudorandom functions from variable-density LPN. In *61st FOCS*, pages 1069–1080. IEEE Computer Society Press, November 2020.
- [BCG⁺20b] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416. Springer, Cham, August 2020.
- [BCG⁺21] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 871–900. Springer, Cham, October 2021.
- [BCG⁺22] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. Correlated pseudorandomness from expand-accumulate codes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 603–633. Springer, Cham, August 2022.
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.
- [BCM⁺24] Dung Bui, Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Fast public-key silent OT and more from constrained Naor-Reingold. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 88–118. Springer, Cham, May 2024.
- [BGI19] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part I*, volume 11891 of *LNCS*, pages 341–371. Springer, Cham, December 2019.
- [BHKL18] Assi Barak, Martin Hirt, Lior Koskas, and Yehuda Lindell. An end-to-end system for large scale P2P MPC-as-a-service and low-bandwidth MPC for weak participants. In

David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 695–712. ACM Press, October 2018.

- [BJMM12] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 520–536. Springer, Berlin, Heidelberg, April 2012.
- [BKRR25] Thomas Buchsteiner, Karl W. Koch, Dragos Rotaru, and Christian Rechberger. webpdz: Versatile MPC on the web. *IACR Cryptol. ePrint Arch.*, page 487, 2025.
- [BMRS21] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 92–122, Virtual Event, August 2021. Springer, Cham.
- [BØ23] Pierre Briaud and Morten Øygarden. A new algebraic approach to the regular syndrome decoding problem and implications for PCG constructions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 391–422. Springer, Cham, April 2023.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [CD23] Geoffroy Couteau and Clément Ducros. Pseudorandom correlation functions from variable-density LPN, revisited. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 221–250. Springer, Cham, May 2023.
- [CRR21] Geoffroy Couteau, Peter Rindal, and Srinivasan Raghuraman. Silver: Silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 502–534, Virtual Event, August 2021. Springer, Cham.
- [CS14] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 327–350. Springer, Berlin, Heidelberg, May 2014.
- [DIO21] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. In Stefano Tessaro, editor, *ITC 2021*, volume 199 of *LIPICs*, pages 5:1–5:24. Schloss Dagstuhl, July 2021.
- [DLMS14] Yuanxi Dai, Jooyoung Lee, Bart Mennink, and John P. Steinberger. The security of multiple encryption in the ideal cipher model. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 20–38. Springer, Berlin, Heidelberg, August 2014.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.

- [Ds17] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 523–535. ACM Press, October / November 2017.
- [DSZ14] Daniel Demmler, Thomas Schneider, and Michael Zohner. Ad-hoc secure two-party computation on mobile devices using hardware tokens. In Kevin Fu and Jaeyeon Jung, editors, *USENIX Security 2014*, pages 893–908. USENIX Association, August 2014.
- [ES24] Andre Esser and Paolo Santini. Not just regular decoding: Asymptotics and improvements of regular syndrome decoding attacks. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VI*, volume 14925 of *LNCS*, pages 183–217. Springer, Cham, August 2024.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- [GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
- [GPR⁺21] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 395–425, Virtual Event, August 2021. Springer, Cham.
- [GYKW24] Radhika Garg, Kang Yang, Jonathan Katz, and Xiao Wang. Scalable mixed-mode MPC. In *2024 IEEE Symposium on Security and Privacy*, pages 523–541. IEEE Computer Society Press, May 2024.
- [GYW⁺23] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 330–362. Springer, Cham, April 2023.
- [HjLHD22] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 809–826. USENIX Association, August 2022.
- [IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.
- [JIF] JIFF. JavaScript library for building web-based applications that employ secure multiparty computation (MPC). <https://github.com/multiparty/jiff>.
- [KL24] Jiseung Kim and Changmin Lee. Reduce and prange: Revisiting prange’s information set decoding for LPN and RSD. Cryptology ePrint Archive, Report 2024/276, 2024.

- [KOS15] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Actively secure OT extension with optimal overhead. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 724–741. Springer, Berlin, Heidelberg, August 2015.
- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842. ACM Press, October 2016.
- [LLY⁺25] Yi Liu, Junzuo Lai, Peng Yang, Qi Wang, Anjia Yang, Siu-Ming Yiu, and Jian Weng. Highly efficient actively secure two-party computation with one-bit advantage bound. In *2025 IEEE Symposium on Security and Privacy*, pages 2846–2864. IEEE Computer Society Press, May 2025.
- [LWYY24] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. The hardness of LPN over any integer ring and field for PCG applications. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 149–179. Springer, Cham, May 2024.
- [MMT11] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 107–124. Springer, Berlin, Heidelberg, December 2011.
- [NNOB12] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 681–700. Springer, Berlin, Heidelberg, August 2012.
- [OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708. Springer, Cham, October 2021.
- [Pat09] Jacques Patarin. The “coefficients H” technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *SAC 2008*, volume 5381 of *LNCS*, pages 328–345. Springer, Berlin, Heidelberg, August 2009.
- [PR] Lance Roy Peter Rindal. libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.
- [RRT23] Srinivasan Raghuraman, Peter Rindal, and Titouan Tanguy. Expand-convolute codes for pseudorandom correlation generators from LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 602–632. Springer, Cham, August 2023.

- [RS21] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 901–930. Springer, Cham, October 2021.
- [SGRR19] Phillipp Schoppmann, Adrià Gascón, Leonie Reichert, and Mariana Raykova. Distributed vector-OLE: Improved constructions and implementation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1055–1072. ACM Press, November 2019.
- [WM] Web-MPC. Platform for deploying web-based privacy-preserving data surveys using secure multi-party computation (MPC). <https://github.com/multiparty/web-mpc>.
- [WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient Multi-Party computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [WYKW21] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021.
- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.
- [YWL⁺20] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1607–1626. ACM Press, November 2020.

A Proof of Lemma 1

Lemma 1 (Correctness of cGGM-tree accumulation). *For any $\alpha, \omega \in \{0, 1\}^h$ and $\Delta, k \in \mathbb{K}$, it holds that*

$$\Pr \left[\begin{array}{l} \{\text{sum}_0^{(i)}\}_{i \in [h]} \leftarrow \text{cGGM.Setup}(\Delta, k), \\ \forall i \in [h] : \text{sum}_{\bar{\alpha}_i}^{(i)} := \bar{\alpha}_i \cdot \Delta + (-1)^{\bar{\alpha}_i} \cdot \text{sum}_0^{(i)}, \\ k_\alpha := \text{cGGM.PuncSetup}(\alpha, \{\text{sum}_{\bar{\alpha}_i}^{(i)}\}_{i \in [h]}), : \quad x - y = f_{\alpha, \Delta}^{\geq}(\omega) \\ x := \text{cGGM.Acc}(\omega, \Delta, k), \\ y := \text{cGGM.PuncAcc}(\omega, \alpha, k_\alpha) \end{array} \right] = 1.$$

Proof. From an induction, we see $\sum_{j \in [2^{i+1}]} \mathbf{s}_i^{(j)} = \Delta$ for the full binary tree spanned from cGGM.Setup and each $i \in [h]$, and that the punctured binary tree spanned from cGGM.PuncSetup is the same as the full one except for the nodes on the punctured path α , i.e.,

$$\forall i \in [h], j \neq \alpha|_{i+1} \in [2^{i+1}] : \mathbf{s}_i^{(j)} = \mathbf{s}_i^{(j)}, \quad (6)$$

where $\mathbf{s}_i^{(j)}$ and $\mathbf{s}_i^{(j)}$ are the nodes with the same position in the punctured and full binary tree, respectively. For any $i \in [h]$ and the $(i+1)$ -bit prefix $\alpha|_{i+1}$, it follows from $\mathbf{s}_{i+1}^{(2^j)} + \mathbf{s}_{i+1}^{(2^j+1)} = \mathbf{s}_i^{(j)}$ in the full binary tree that

$$\begin{aligned} & \mathbf{s}_i^{(\alpha|_{i+1})} + \sum_{j \in [i+1]} \mathbf{s}_j^{(2(\alpha|_j) + \bar{\alpha}_j)} \\ &= \mathbf{s}_i^{(\alpha|_{i+1})} + \sum_{j \in [2^{i+1}], j \neq \alpha|_{i+1}} \mathbf{s}_i^{(j)} \\ &= \sum_{j \in [2^{i+1}]} \mathbf{s}_i^{(j)} = \Delta. \end{aligned} \quad (7)$$

Now, consider the accumulation. On the one hand, if $\omega \neq \alpha$, it holds that

$$\begin{aligned} x - y &= \left(\sum_{i \in [h]} \mathbb{1}(\omega_i = 1) \cdot \mathbf{s}_i^{(2(\omega|_i) + \bar{\omega}_i)} + \mathbf{s}_{h-1}^{(\omega)} \right) \\ &\quad - \sum_{i \in [p]} \mathbb{1}(\omega_i = 1) \cdot \mathbf{s}_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \\ &\quad + \mathbb{1}(\omega_p = 1) \cdot \left(\sum_{i \in [p+1]} \mathbf{s}_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \right) \\ &\quad - \sum_{i \in [p+1, h-1]} \mathbb{1}(\omega_i = 1) \cdot \mathbf{s}_i^{(2(\omega|_i) + \bar{\omega}_i)} \\ &\quad - \mathbf{s}_{h-1}^{(\omega)} \\ &= \mathbb{1}(\omega_p = 1) \cdot \left(\mathbf{s}_p^{(\alpha|_{p+1})} + \sum_{i \in [p+1]} \mathbf{s}_i^{(2(\alpha|_i) + \bar{\alpha}_i)} \right) \\ &= \mathbb{1}(\omega_p = 1) \cdot \Delta = f_{\alpha, \Delta}^{\geq}(\omega). \end{aligned}$$

On the other hand, if $\omega = \alpha$, we have

$$\begin{aligned} x - y &= \left(\sum_{i \in [h]} \mathbb{1}(\omega_i = 1) \cdot \mathbf{s}_i^{(2(\omega|_i) + \bar{\omega}_i)} + \mathbf{s}_{h-1}^{(\omega)} \right) \\ &\quad - \left(\sum_{i \in [h]} \mathbb{1}(\omega_i = 1) \cdot \mathbf{s}_i^{(2(\alpha|_i) + \bar{\alpha}_i)} + \mathbf{s}_{h-1}^{(\alpha)} \right) \\ &= \mathbf{s}_{h-1}^{(\alpha)} + \sum_{j \in [h]} \mathbf{s}_j^{(2(\alpha|_j) + \bar{\alpha}_j)} \\ &= \Delta = f_{\alpha, \Delta}^{\geq}(\omega). \end{aligned}$$

Putting the two cases together, this lemma is proved. $\square$

B Background on H-coefficient Technique

In this appendix, we briefly introduce the H-coefficient technique to be used in our proofs in the following appendices. H-coefficient technique [Pat09, CS14] can be used to bound the advantage of an information-theoretic adversary. We borrow the summary of the H-coefficient technique from [GKWY20] and refer readers to [CS14] for a comprehensive introduction. Let $\mathcal{T}$ denote the set of *attainable* transcripts for an information-theoretic distinguisher, and let $\Pr_{\text{real}}[\cdot]$ and $\Pr_{\text{ideal}}[\cdot]$ be the probabilities in the real and ideal worlds, respectively. The H-coefficient technique divides $\mathcal{T}$ into a “bad” subset $\mathcal{T}_{\text{bad}}$ and a “good” subset $\mathcal{T}_{\text{good}} := \mathcal{T} \setminus \mathcal{T}_{\text{bad}}$, and shows that if it holds that

$$\begin{aligned} \Pr_{\text{ideal}}[\mathcal{Q} \in \mathcal{T}_{\text{bad}}] &\leq \epsilon_1, \\ \forall \mathcal{Q} \in \mathcal{T}_{\text{good}} : \frac{\Pr_{\text{real}}[\mathcal{Q}]}{\Pr_{\text{ideal}}[\mathcal{Q}]} &\geq 1 - \epsilon_2, \end{aligned}$$

then the advantage of the distinguisher is at most $\epsilon_1 + \epsilon_2$.

C Proof of Theorem 1

Theorem 1. *In the $(\mathcal{F}_{\text{sVOLE}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{EQ}})$ -hybrid model and under the EA-LPN-SL($t, n, N, \ell, \mathbb{F}$) assumption, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any malicious adversary, if hash function $H(x) = \pi(\sigma(x)) + \sigma(x)$, where $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a (programmable) random permutation and $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an efficient linear orthomorphism.*

Proof. This theorem is from Lemma 2 and Lemma 3. $\square$

Lemma 2. *In the $(\mathcal{F}_{\text{sVOLE}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{EQ}})$ -hybrid model and under the EA-LPN-SL($t, n, N, \ell, \mathbb{F}$) assumption, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any malicious sender P_0 .*

Proof. Consider the following hybrid argument.

- **Hybrid₀.** This is the real-world execution.
- **Hybrid₁.** This hybrid is identical to the previous one, except that there is a machine $\mathcal{S}$ playing as the honest P_1 and emulating the subroutine functionality $\mathcal{F}_{\text{sVOLE}}$ (where $\mathcal{S}$ can extract Δ and $\{K(s_i)\}_{i \in [t]}$), $\mathcal{F}_{\text{Rand}}$, and $\mathcal{F}_{\text{EQ}}$. This change is conceptual so that these two hybrids are perfectly indistinguishable.
- **Hybrid₂.** This hybrid is identical to the previous one, except that $\mathcal{S}$ changes the way to compute the outputs of the honest P_1 . In the **Initialize** phase, $\mathcal{S}$ receives $\mathbf{c}_i$ from the corrupted P_0 for each $i \in [t]$, and proceeds as follows in the consistency check:
 1. $\mathcal{S}$ extracts $K(\mathbf{m})$ from the emulated $\mathcal{F}_{\text{sVOLE}}$, samples $\mathbf{z} \leftarrow \mathbb{F}^\mu$, and sends $\mathbf{z}$ to the corrupted P_0 .
 2. $\mathcal{S}$ extracts v from the emulated $\mathcal{F}_{\text{EQ}}$.
 3. For each $i \in [t]$ and $\alpha'_i \in \{0, 1\}^h = [\frac{N}{t}]$, $\mathcal{S}$ defines the following cGGM results as in the protocol:

$$\begin{aligned} \mathbf{r}_i &:= (\overline{\alpha}'_{i,0}, \dots, \overline{\alpha}'_{i,h-1}), \\ M(\mathbf{r}_i) &:= -K(\mathbf{s}_i^{(1,h)}) - \mathbf{d}_i \cdot \Delta + \mathbf{r}_i \cdot K(\mathbf{s}_i^{(0)}), \\ \{\text{sum}_{i,\alpha'_i,j}^{(j)}\}_{j \in [h]} &:= ((-1)^{\mathbf{r}_i^{(0)}}, \dots, (-1)^{\mathbf{r}_i^{(h-1)}}) \odot (-M(\mathbf{r}_i) + \mathbf{c}_i), \\ k_{i,\alpha'_i} &:= \text{cGGM.PuncSetup}(\alpha'_i, \{\text{sum}_{i,\alpha'_i,j}^{(j)}\}_{j \in [h]}). \end{aligned} \tag{8}$$

Let $\alpha' := (\alpha'_0, \dots, \alpha'_{t-1})$. $\mathcal{S}$ uses the cGGM results to behave as an honest P_1 to compute its α' -dependent consistency check input $(e_{\alpha'}, M(e_{\alpha'}))$ and

$$w_{\alpha'} := U(M(e_{\alpha'}) - e_{\alpha'} \cdot \Delta) + \sum_{i \in [\mu]} (K(m^{(i)}) + z^{(i)} \cdot \Delta) \cdot X^i. \quad (9)$$

4. $\mathcal{S}$ computes a set $\mathcal{I}_v \subseteq [\frac{N}{t}]^t$ such that $\mathcal{I}_v = \{\alpha' \in [\frac{N}{t}]^t \mid w_{\alpha'} = v\}$. If the real $\alpha = (\alpha_0, \dots, \alpha_{t-1})$ used by P_1 satisfies $\alpha \notin \mathcal{I}_v$, $\mathcal{S}$ outputs 0 in the emulated $\mathcal{F}_{\text{EQ}}$ and aborts P_1 .
5. If $\mathcal{S}$ does not abort P_1 , $\mathcal{S}$ uses a random $\alpha' \leftarrow \mathcal{I}_v$ as in (8) to define t “modified” cGGM expansion vectors such that, for each $i \in [t]$ and the i -th tree,
 - Leaf nodes not on the path α'_i are defined as in

$$\text{cGGM.PuncSetup}(\alpha'_i, \{\text{sum}_{i, \alpha'_{i,j}}^{(j)}\}_{j \in [h]}).$$

- The on-path node $s_{h-1}^{(\alpha'_i)}$ is alternatively set as

$$s_{h-1}^{(\alpha'_i)} := M(\beta_i) - \sum_{j \in [2h], j \neq \alpha'_i} s_{h-1}^{(j)} - \beta_i \cdot \Delta, \quad (10)$$

instead of being defined in the previous hybrid as

$$s_{h-1}^{(\alpha'_i)} := M(\beta_i) - \sum_{j \in [2h], j \neq \alpha'_i} s_{h-1}^{(j)}.$$

In each **Extend** phase, $\mathcal{S}$ uses the same randomness with the corrupted P_0 to sample $\mathbf{b}$ and run the accumulation on the t “modified” cGGM expansion vectors to obtain $K(x_{\alpha'})$, and computes from the real α (instead of α')

$$\begin{aligned} x &:= \sum_{i \in [t], j \in \mathcal{J}} \mathbf{b}^{(j)} \cdot \mathbb{1}(j \geq i \cdot \frac{N}{t} + \alpha_i) \cdot \beta_i, \\ M(x) &:= K(x_{\alpha'}) + x \cdot \Delta. \end{aligned}$$

Finally, $\mathcal{S}$ asks P_1 to output $(x, M(x))$.

We prove that the two hybrids are statistically indistinguishable as follows.

First, the view of the adversary includes $\{\mathbf{d}_i\}_i$, $\mathbf{z}$, and the outputs of $\mathcal{F}_{\text{svOLE}}$, $\mathcal{F}_{\text{Rand}}$, and $\mathcal{F}_{\text{EQ}}$. $\{\mathbf{d}_i\}_i$ are identically defined in both two hybrids. Moreover, $\mathbf{z}$ are uniform due to (i) the uniform one-time pad $\mathbf{m}$ from $\mathcal{F}_{\text{svOLE}}$ in the previous hybrid, or (ii) the uniform sampling of $\mathcal{S}$ in this hybrid. The outputs of all functionalities are also identically distributed, where the output of $\mathcal{F}_{\text{EQ}}$ has the same distribution due to the definition of $\mathcal{I}_v$ and the abort condition in step 4. In other words, the view of the adversary has the same distribution in the two hybrids.

Second, conditioned on the identical view of the adversary, the joint distribution are statistically indistinguishable. On the one hand, consider the case where P_1 aborts without an output. In the previous hybrid, P_1 aborts if and only if $w \neq v$ in $\mathcal{F}_{\text{EQ}}$, where

$$\begin{aligned} w &= U(M(e_{\alpha})) + \sum_{i \in [\mu]} M(m^{(i)}) \cdot X^i \\ &= U(M(e_{\alpha})) + \sum_{i \in [\mu]} (K(m^{(i)}) + m^{(i)} \cdot \Delta) \cdot X^i \\ &= U(M(e_{\alpha})) - \sum_{i \in [\mu]} u^{(i)} \cdot X^i \cdot \Delta + \sum_{i \in [\mu]} (K(m^{(i)}) + z^{(i)} \cdot \Delta) \cdot X^i \\ &= U(M(e_{\alpha})) - U(e_{\alpha}) \cdot \Delta + \sum_{i \in [\mu]} (K(m^{(i)}) + z^{(i)} \cdot \Delta) \cdot X^i \\ &= U(M(e_{\alpha}) - e_{\alpha} \cdot \Delta) + \sum_{i \in [\mu]} (K(m^{(i)}) + z^{(i)} \cdot \Delta) \cdot X^i, \end{aligned}$$

which has the same distribution as w_α defined in this hybrid. In this hybrid, P_1 aborts if and only if $\alpha \notin \mathcal{I}_v$, or equivalently, $w_\alpha \neq v$. Therefore, the abort condition is identical in the two hybrids and occurs with the same probability.

On the other hand, consider the complement case where P_1 has an output. We stress that x in this output is identically defined in both two hybrids. So, consider $M(x)$. In the previous hybrid, $M(x)$ is computed from the punctured cGGM trees defined from the keys in (8) for $\alpha' = \alpha$. In this hybrid, $M(x)$ is computed from random $\alpha' \leftarrow \mathcal{I}_v$ and the identically defined punctured cGGM trees for α' except for the on-path nodes as per (10).

In this hybrid, we can prove that for $\alpha, \alpha' \in \mathcal{I}_v$ (where $\alpha \in \mathcal{I}_v$ is implied by that P_1 does not abort), $K(x_\alpha) = K(x_{\alpha'})$ except with probability ϵ . Since $\alpha = \alpha'$ is the trivial case, we focus on two distinct $\alpha, \alpha' \in \mathcal{I}_v$. From the definition of $\mathcal{I}_v$, it holds that $w_\alpha = w_{\alpha'} = v$. From (9), we know that $w_\alpha = w_{\alpha'}$ is equivalent to

$$U(M(e_\alpha) - e_\alpha \cdot \Delta) = U(M(e_{\alpha'}) - e_{\alpha'} \cdot \Delta).$$

Since U is a linear ϵ -universal hash, for each $i \in [t]$ and $d \in [\frac{N}{t}]$, it holds that

$$\begin{aligned} & M(e_\alpha^{(i \cdot \frac{N}{t} + d)}) - e_\alpha^{(i \cdot \frac{N}{t} + d)} \cdot \Delta \\ &= M(e_{\alpha'}^{(i \cdot \frac{N}{t} + d)}) - e_{\alpha'}^{(i \cdot \frac{N}{t} + d)} \cdot \Delta \end{aligned} \quad (11)$$

except with probability ϵ . Consider some fixed i and the case where (11) holds. If $\alpha_i = \alpha'_i$, the accumulation for $K(x_\alpha), K(x_{\alpha'})$ in the i -th tree is trivially identical. We assume $\alpha_i \neq \alpha'_i$ for non-triviality. For each $d \neq \alpha_i, \alpha'_i$, we have $e_\alpha^{(i \cdot \frac{N}{t} + d)} = e_{\alpha'}^{(i \cdot \frac{N}{t} + d)} = 0$ and thus, in the two i -th trees for α and α' , the d -th leaf node

$$M(e_\alpha^{(i \cdot \frac{N}{t} + d)}) = M(e_{\alpha'}^{(i \cdot \frac{N}{t} + d)}).$$

For $d = \alpha_i$ or $d = \alpha'_i$, we have the leaf nodes

$$\begin{aligned} M(e_{\alpha'}^{(i \cdot \frac{N}{t} + \alpha_i)}) &= M(e_\alpha^{(i \cdot \frac{N}{t} + \alpha_i)}) - \beta_i \cdot \Delta, \\ M(e_\alpha^{(i \cdot \frac{N}{t} + \alpha'_i)}) &= M(e_{\alpha'}^{(i \cdot \frac{N}{t} + \alpha'_i)}) - \beta_i \cdot \Delta. \end{aligned}$$

Since $K(x_\alpha)$ is accumulated from the same t “modified” cGGM expansion vectors as $M(e_\alpha)$ except for the difference (10) of the leaf nodes in α , the j -th term (where $j = \mu_j \cdot \frac{N}{t} + \omega_j$ and $\mu_j = i$) from the i -th tree is

$$\begin{aligned} & \sum_{d \leq \omega_j, d \neq \alpha_i} M(e_\alpha^{(i \cdot \frac{N}{t} + d)}) + \mathbb{1}(\omega_j \geq \alpha_i) \cdot (M(e_\alpha^{(i \cdot \frac{N}{t} + \alpha_i)}) - \beta_i \cdot \Delta) \\ &= \sum_{d \leq \omega_j, d \neq \alpha_i} M(e_\alpha^{(i \cdot \frac{N}{t} + d)}) + \mathbb{1}(\omega_j \geq \alpha_i) \cdot M(e_{\alpha'}^{(i \cdot \frac{N}{t} + \alpha_i)}) \\ &= \sum_{d \leq \omega_j, d \neq \alpha_i, \alpha'_i} M(e_{\alpha'}^{(i \cdot \frac{N}{t} + d)}) + \mathbb{1}(\omega_j \geq \alpha'_i) \cdot M(e_\alpha^{(i \cdot \frac{N}{t} + \alpha'_i)}) + \mathbb{1}(\omega_j \geq \alpha_i) \cdot M(e_{\alpha'}^{(i \cdot \frac{N}{t} + \alpha_i)}) \\ &= \sum_{d \leq \omega_j, d \neq \alpha'_i} M(e_{\alpha'}^{(i \cdot \frac{N}{t} + d)}) + \mathbb{1}(\omega_j \geq \alpha'_i) \cdot (M(e_{\alpha'}^{(i \cdot \frac{N}{t} + \alpha'_i)}) - \beta_i \cdot \Delta), \end{aligned}$$

which is the j -th term from the i -th tree for $K(x_{\alpha'})$. For all other j 's with $\mu_j \neq i$, the j -th term for $K(x_\alpha)$ and $K(x_{\alpha'})$ are 0. Putting these cases together, we have that for $\alpha, \alpha' \in \mathcal{I}_v$, $K(x_\alpha) = K(x_{\alpha'})$ conditioned on (11).

As a result, we can write by definition

$$\begin{aligned} M(x) &= K(x_{\alpha'}) + x \cdot \Delta \\ &= K(x_\alpha) + x_\alpha \cdot \Delta = M(x_\alpha), \end{aligned}$$

where $M(x_\alpha)$ in this hybrid is exactly the $M(x)$ computed from the punctured cGGM trees defined from the keys in (8) for $\alpha' = \alpha$ as in the previous hybrid. This concludes that, in both two hybrids, the output of P_1 has the same distribution conditioned on the same view of the adversary and that P_1 has an output, except with probability ϵ .

Putting all things together, this hybrid is statistically indistinguishable from the previous one with an advantage at most ϵ , which is negligible.

- **Hybrid₃.** This hybrid is identical to the previous one, except that $\mathcal{S}$ sends $K(x_{\alpha'})$ in each **Extend** phase to the functionality to define the output of the honest P_1 . This hybrid is the ideal world and indistinguishable from the previous one due to the LPN assumption with static leakage with respect to the adversarial guess $\mathcal{I}_v$.

The above hybrid argument concludes this theorem. $\square$

Lemma 3. *In the $(\mathcal{F}_{\text{sVOLE}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{EQ}})$ -hybrid model, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any malicious receiver P_1 if hash function $H(x) = \pi(\sigma(x)) + \sigma(x)$, where $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a (programmable) random permutation and $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an efficiently computable linear orthomorphism.*

Proof. When the receiver is corrupted, the view of the malicious adversary includes $\{c_i\}_i$ and the outputs of $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{F}_{\text{Rand}}$, and $\mathcal{F}_{\text{EQ}}$ (and the random permutation for H calls in cGGM). In the simulation, the simulator simply sends uniformly random $\{c_i\}_i$ to the corrupted P_1 , emulates $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{F}_{\text{Rand}}$, and the random permutation on-the-fly, and aborts P_0 in the emulated $\mathcal{F}_{\text{EQ}}$ if some d_i sent by the corrupted P_1 contains a malformed $r_i \notin \{0, 1\}^h \subseteq \mathbb{F}^h$ if $\beta_i \neq 0$. Moreover, following the simulation strategy of [GYW⁺23, Theorem 7, full version] against the corrupted receiver, the simulator observes each query of the adversary to the random permutation to compute a guess Δ' , sends Δ' to the functionality to check whether $\Delta' = \Delta$, and programs the random permutation to be consistent with $\{c_i\}_i$ and the correct guess Δ . We can bound the advantage of the adversary via the H-coefficient technique [Pat09, CS14, DLMS14] as in [GYW⁺23]. This advantage is still negligible. The detailed proof is as follows.

When the receiver is corrupted, the simulator $\mathcal{S}$ in the ideal world emulates the random permutation π and its inverse π^{-1} throughout the lifespan of the distinguishing game. During the distinguishing game in either world, let $\mathcal{Q}_\pi$ denote an *ordered* list of the adversary's query-answer pairs of the random permutation or its inverse (note that $(x, y) \in \mathcal{Q}_\pi$ means that the adversary learns $\pi(x) = y$, regardless of whether it queried $\pi(x)$ or $\pi^{-1}(y)$), and $\mathcal{Q}_\mathcal{O}$ denote an *ordered* list of the adversary's query-answer pairs of the “oracle” emulated by the non-corrupted machine(s) (i.e., the honest P_0 plus the subroutine functionalities $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{F}_{\text{Rand}}$, and $\mathcal{F}_{\text{EQ}}$ in the real world, or the simulator $\mathcal{S}$ in the ideal world). One can think that, for each $i \in [t]$ execution in the **Initialize** phase, $\mathcal{Q}_\mathcal{O}$ records the query $(\beta_i, M(\beta_i), r_i, M(r_i))$ (i.e., the effective sVOLE output committed by the corrupted P_1 , see below) with its answer c_i (i.e., the transcript sent by the honest P_0). $\mathcal{S}$ has access to $\mathcal{Q}_\pi$ and $\mathcal{Q}_\mathcal{O}$ in the ideal world.

$\mathcal{S}$ sends (init) to $\mathcal{F}_{\text{sPCG-sVOLE}}$ upon receiving the first (init) from $\mathcal{A}$ to $\mathcal{F}_{\text{sVOLE}}$, and works as follows for each $i \in [t]$ parallel execution in the **Initialize** phase:

1. Upon receiving (extend, $h + 1$) from $\mathcal{A}$ to $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{S}$ waits for $\mathcal{A}$ to choose its sVOLE output $(s_i, M(s_i))$.
2. $\mathcal{S}$ extracts $(\beta_i, M(\beta_i))$, receives d_i from $\mathcal{A}$, and
 - (a) If $\beta_i = 0$, picks a random $r_i \leftarrow \{0, 1\}^h$.

(b) If $\beta_i \neq 0$, extracts $\mathbf{r}_i := \beta_i^{-1} \cdot (\mathbf{d}_i - \mathbf{s}_i^{(1,h)}) \in \mathbb{F}^h$.

3. $\mathcal{S}$ computes $\mathbf{M}(\mathbf{r}_i) := \mathbf{r}_i \cdot \mathbf{M}(\beta_i) - \mathbf{M}(\mathbf{s}_i^{(1,h)})$.

4. $\mathcal{S}$ sends random $\mathbf{c}_i$ to $\mathcal{A}$ and appends $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i)$ to $\mathcal{Q}_{\mathcal{O}}$.

In the consistency check in the **Initialize** phase:

1. Upon receiving (extend, μ) from $\mathcal{A}$ to $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{S}$ waits for $\mathcal{A}$ to choose its sVOLE output $(\mathbf{m}, \mathbf{M}(\mathbf{m}))$. Moreover, $\mathcal{S}$ emulates $\mathcal{F}_{\text{Rand}}$ to output a random $\mathbf{U}$ to $\mathcal{A}$.
2. $\mathcal{S}$ receives $\mathbf{z}'$ from $\mathcal{A}$.
3. If there exists $i \in [t]$ such that $\beta_i \neq 0$ and the extracted $\mathbf{r}_i \notin \{0, 1\}^h$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{EQ}}$ to output 0 to $\mathcal{A}$ and aborts; otherwise, continues.
4. $\mathcal{S}$ computes $(\mathbf{e}, \mathbf{M}(\mathbf{e}), \mathbf{z}, w)$ as an honest $\mathbf{P}_1$ in the protocol and extracts w' from the emulated $\mathcal{F}_{\text{EQ}}$. If $\mathbf{z}' \neq \mathbf{z}$ or $w' \neq w$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{EQ}}$ to output 0 to $\mathcal{A}$ and aborts; otherwise, $\mathcal{S}$ emulates $\mathcal{F}_{\text{EQ}}$ to output 1 to $\mathcal{A}$.

In the **Extend** phase, if $\mathcal{S}$ does not abort:

1. $\mathcal{S}$ uses the same randomness to sample $\mathbf{b}$ and computes $(x, \mathbf{M}(x))$ as an honest $\mathbf{P}_1$ in the protocol.
2. $\mathcal{S}$ sends (extend) and $(x, \mathbf{M}(x))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$.

If $\mathcal{S}$ does not abort, $\mathcal{S}$ invokes **Global-key queries** of $\mathcal{F}_{\text{sPCG-sVOLE}}$ in the following cases:

- query₁: $\mathcal{Z}$ queries the random permutation π with x or its inverse π^{-1} with y .

For each $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i) \in \mathcal{Q}_{\mathcal{O}}$ where $\beta_i \neq 0$:

1. $\mathcal{S}$ computes

$$\begin{aligned} \alpha_i &:= \alpha_{i,0} \dots \alpha_{i,h-1} := \bar{\mathbf{r}}_i^{(0)} \dots \bar{\mathbf{r}}_i^{(h-1)} \in \{0, 1\}^h, \\ \{z_{i,j}\}_{j \in [h]} &:= \text{cGGM.OffPath}(\alpha_i, ((-1)^{\mathbf{r}_i^{(0)}}, \dots, (-1)^{\mathbf{r}_i^{(h-1)}}) \odot (-\mathbf{M}(\mathbf{r}_i) + \mathbf{c}_i)), \quad (12) \\ \forall k \in [1, h-1] : w_{i,k} &:= \sum_{j \in [k]} z_{i,j}, \end{aligned}$$

where the macro cGGM.OffPath , on input a path $\alpha_i \in \{0, 1\}^h$ and h sums $\{\text{sum}_{i, \bar{\alpha}_{i,j}}^{(j)}\}_{j \in [h]}$ used in cGGM.PuncSetup to define an h -level correlated GGM tree except the h on-path nodes, outputs the h off-path nodes for the path α_i . This macro ensures that, for $j \in [h]$,

$$\begin{aligned} z_{i,j} &= (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{M}(\mathbf{r}_i^{(j)}) + \mathbf{c}_i^{(j)}) - \text{“other } \mathbf{r}_i^{(j)}\text{-side nodes on the } j\text{-th level} \\ &\quad \text{defined by the off-path nodes } \{z_{i,k}\}_{k \in [j]} \text{”}. \end{aligned}$$

2. For each $j \in [1, h-1]$, $\mathcal{S}$ extracts $\mathbf{K}(\beta_i)'_1 \in \mathbb{K}$ and sends $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_1))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$ if $\mathcal{Z}$ queries π with x , or extracts $\mathbf{K}(\beta_i)'_2 \in \mathbb{K}$ and $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_2))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$ if $\mathcal{Z}$ queries π^{-1} with y , where

$$\begin{aligned} \mathbf{K}(\beta_i)'_1 &:= \sigma^{-1}(x) + w_{i,j}, \\ \mathbf{K}(\beta_i)'_2 &:= \begin{cases} \sigma^{-1}(-y + z_{i,j}) + w_{i,j} & , \text{ if } \mathbf{r}_i^{(j)} = 0 \\ \sigma'^{-1}(-y - z_{i,j}) + w_{i,j} & , \text{ if } \mathbf{r}_i^{(j)} = 1 \end{cases} \quad (13) \end{aligned}$$

Note that $\sigma'(x) := \sigma(x) - x$ is a permutation as $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an orthomorphism, and its inverse σ'^{-1} should be well-defined.

- **query₂**: A new pair $((\beta_i, M(\beta_i), \mathbf{r}_i, M(\mathbf{r}_i)), \mathbf{c}_i)$ where $\beta_i \neq 0$ is added to $\mathcal{Q}_{\mathcal{O}}$ during execution.

From this pair, $\mathcal{S}$ computes $\{z_{i,j}\}_{j \in [h]}$ and $\{w_{i,j}\}_{j \in [1, h-1]}$ as per (12). Then, for each $(x, y) \in \mathcal{Q}_{\pi}$ and each $j \in [1, h-1]$, $\mathcal{S}$ extracts both $K(\beta_i)'_1$ and $K(\beta_i)'_2$ as in (13), and sends two global-key guesses $(\text{guess}, \beta_i^{-1} \cdot (M(\beta_i) - K(\beta_i)'_1))$ and $(\text{guess}, \beta_i^{-1} \cdot (M(\beta_i) - K(\beta_i)'_2))$ to $\mathcal{F}_{\text{PCG-sVOLE}}$.

In either case, if $\mathcal{S}$ receives (**success**) from $\mathcal{F}_{\text{PCG-sVOLE}}$ for some guess Δ , $\mathcal{S}$ will program the random permutation π and its inverse π^{-1} such that, for each $i \in [t]$, the offset $K(\beta_i) := M(\beta_i) - \beta_i \cdot \Delta$, the $\{z_{i,j}\}_{j \in [h]}$ and $\{w_{i,j}\}_{j \in [1, h-1]}$ computed from the i -th $((\beta_i, M(\beta_i), \mathbf{r}_i, M(\mathbf{r}_i)), \mathbf{c}_i) \in \mathcal{Q}_{\mathcal{O}}$, and each $j \in [1, h-1]$,

$$\pi(\sigma(K(\beta_i) - w_{i,j})) = -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j}), \quad (14)$$

which must hold in the real world due to the construction of $H(x) = \pi(\sigma(x)) + \sigma(x)$ and the definition of $z_{i,j}$ and $w_{i,j}$. That is, the programming artificially ensures the real-world consistency among the off-path nodes on the cGGM tree in the ideal world.

Below, we prove that the two worlds are statistically indistinguishable. First, we consider the event that the honest P_0 aborts. In the real world, this event occurs if and only if $w' \neq v$, where w' is the value sent from $\mathcal{A}$ to $\mathcal{F}_{\text{EQ}}$. According to the protocol, it holds that

$$\begin{aligned} v &= U(K(e)) + \sum_{k \in [\mu]} (K(\mathbf{m}^{(k)}) + \mathbf{z}'^{(k)} \cdot \Delta) \cdot \mathbf{X}^k \\ &= U(K(e)) + \sum_{k \in [\mu]} (M(\mathbf{m}^{(k)}) + (\mathbf{z}'^{(k)} - \mathbf{m}^{(k)}) \cdot \Delta) \cdot \mathbf{X}^k. \end{aligned}$$

If there exists $i \in [t]$ such that $\beta_i \neq 0$ and $\mathbf{r}_i := \beta_i^{-1} \cdot (\mathbf{d}_i - \mathbf{s}_i^{(1,h)}) \notin \{0, 1\}^h$, there must exist some $j \in [h]$ such that $\mathbf{r}_i^{(j)} \notin \{0, 1\}$ and $\alpha_i|_j = \bar{\mathbf{r}}_i^{(0)} \dots \bar{\mathbf{r}}_i^{(j-1)} \in \{0, 1\}^j$. For this (i, j) , $\mathcal{A}$ can compute

$$-M(\mathbf{r}_i^{(j)}) + \mathbf{c}_i^{(j)} = \text{sum}_{i,0}^{(j)} - \mathbf{r}_i^{(j)} \cdot K(\beta_i). \quad (15)$$

Meanwhile, to ensure $w' = v$, $\mathcal{A}$ must guess the value of

$$U(K(e)) + (\sum_{k \in [\mu]} (\mathbf{z}'^{(k)} - \mathbf{m}^{(k)}) \cdot \mathbf{X}^k) \cdot \Delta.$$

According to the construction of U , $U(K(e))$ contains an additive term, which is *the linear combination of the leaf nodes of the sub-tree rooted in the following j -th level off-path node (located at the $b \in \{0, 1\}$ side) in the i -th cGGM tree of the honest sender P_0* :

$$\begin{aligned} \mathbf{s}_{i,j}^{(2(\alpha_i|_j)+b)} &= \text{sum}_{i,b}^{(j)} - \sum_{k \in [2^j], k \neq \alpha_i|_j} \mathbf{s}_{i,j}^{(2k+b)} \\ &= (-1)^b \cdot (\text{sum}_{i,0}^{(j)} - b \cdot K(\beta_i)) - \sum_{k \in [2^j], k \neq \alpha_i|_j} \mathbf{s}_{i,j}^{(2k+b)}, \end{aligned} \quad (16)$$

where all other nodes on the right hand can be computed by $\mathcal{A}$ via the cGGM-tree expansion of the previous levels. When this node is in the view of $\mathcal{A}$, it can essentially solve $K(\beta_i)$ from (15) and (16) and guess $\Delta := \beta_i^{-1} \cdot (M(\beta_i) - K(\beta_i))$. However, this can only happen with probability at most $\frac{1}{|\mathbb{K}|}$ due to the randomness of Δ (note that Δ has not been revealed to $\mathcal{Z}$ yet). When this node is not in the view of $\mathcal{A}$, an induction can show that $\mathcal{A}$ cannot guess a leaf node (without loss of generality, the left-most leaf node) on its sub-tree except with at most a negligible probability due to the randomness of the random permutation over $\mathbb{K}$. Except with probability $\frac{1}{|\mathbb{K}|}$ that the random coefficient to combine this node is zero, the guess $w' = v$ can be equivalently written as that this node equals to some value *independent of this node*. This only occurs with the negligible probability due to the entropy of this node from the view of $\mathcal{A}$. Putting all things together, in the real world, $\mathcal{F}_{\text{EQ}}$ outputs 0 and the honest P_0 aborts with probability at least $1 - \frac{1}{|\mathbb{K}|} - \text{negl}(\lambda)$, if

there exists $i \in [t]$ such that $\beta_i \neq 0$ and $\mathbf{r}_i := \beta_i^{-1} \cdot (\mathbf{d}_i - \mathbf{s}_i^{(1,h)}) \notin \{0, 1\}^h$. In contrast, in the ideal world, $\mathcal{F}_{\text{EQ}}$ outputs 0 and the honest P_0 aborts with probability 1 in the same case.

Otherwise, for all extracted well-defined $\mathbf{r}_i \in \{0, 1\}^h$, their resulting honestly computed $(\mathbf{z}, w)$ (note that $\beta_i = 0$ does not affect correctness), and the received or extracted $(\mathbf{z}', w')$, if $\mathbf{z}' \neq \mathbf{z}$ or $w' \neq w$, $\mathcal{A}$ has to guess $\Delta \in \mathbb{K}$ to ensure $w' = v$ in the real world. This guess fails so that $\mathcal{F}_{\text{EQ}}$ outputs 0 to abort P_0 with probability at least $1 - \frac{1}{|\mathbb{K}|}$. In contrast, in the ideal world, $\mathcal{F}_{\text{EQ}}$ outputs 0 and the honest P_0 aborts with probability 1 in the same case.

The above argument shows that the abort event happens with statistically close probabilities in the two worlds. Then, we consider the case conditioned on that this event does not happen.

Conditioned on the event that $\mathcal{F}_{\text{EQ}}$ outputs 1 and P_0 does not abort, we use the H-coefficient technique to prove that the ideal world is statistically indistinguishable from the real one. Without loss of generality, we consider a deterministic environment $\mathcal{Z}$ and the transcript $\mathcal{Q}$, which is a part of the joint distribution of the view of $\mathcal{A}$ and the output of the honest P_0 (note that the remaining part of this joint distribution is the $K(x)$'s in all **Extend** executions, and its consistency with $\mathcal{Q}$ will be checked in the end).

A transcript $\mathcal{Q} = (\Delta, \mathcal{Q}_\pi, \mathcal{Q}_\mathcal{O})$ is **bad** if it falls into one of the following cases:

- **bad₁**. There exist two distinct transcript pairs

$$\begin{aligned} & \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_\mathcal{O} \times [1, h-1], \\ & \left(((\beta_{i'}, \mathbf{M}(\beta_{i'}), \mathbf{r}_{i'}, \mathbf{M}(\mathbf{r}_{i'})), \mathbf{c}_{i'}), j' \right) \in \mathcal{Q}_\mathcal{O} \times [1, h-1] \end{aligned}$$

such that, for $K(\beta_i) := \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$ and $K(\beta_{i'}) := \mathbf{M}(\beta_{i'}) - \beta_{i'} \cdot \Delta$, it holds that

$$\begin{aligned} & \left(K(\beta_i) - w_{i,j} = K(\beta_{i'}) - w_{i',j'} \right) \\ \vee & \left(\begin{aligned} & -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j}) \\ & = -\sigma(K(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (K(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{aligned} \right), \end{aligned}$$

where $z_{i,j}, w_{i,j}, z_{i',j'}, w_{i',j'}$ are defined as per (12). This bad case captures the collision between the queries to the random permutation and its inverse.

- **bad₂**. There exists a transcript pair

$$\left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_\mathcal{O} \times [1, h-1]$$

such that, for $K(\beta_i) := \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$, it holds that

$$\begin{aligned} & \left(\begin{aligned} & (\sigma(K(\beta_i) - w_{i,j}), \dots) \in \mathcal{Q}_\pi \\ \vee & (\dots, -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j})) \in \mathcal{Q}_\pi \end{aligned} \right) \\ & \wedge \left(\pi(\sigma(K(\beta_i) - w_{i,j})) \neq -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j}) \right), \end{aligned}$$

where $z_{i,j}, w_{i,j}, z_{i',j'}, w_{i',j'}$ are defined as per (12). This bad case captures the inconsistency in the already defined entries of the random permutation and its inverse.

Consider bad_1 in the ideal world. For any two distinct pairs in $\mathcal{Q}_O \times [1, h-1]$,

$$\begin{aligned}
& \Pr_{\text{ideal}} \left[\bigvee \left(\begin{array}{l} \left(\mathbf{K}(\beta_i) - w_{i,j} = \mathbf{K}(\beta_{i'}) - w_{i',j'} \right) \\ -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \\ = -\sigma(\mathbf{K}(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (\mathbf{K}(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{array} \right) \right] \\
& \leq \Pr_{\text{ideal}} [\mathbf{K}(\beta_i) - w_{i,j} = \mathbf{K}(\beta_{i'}) - w_{i',j'}] \\
& \quad + \Pr_{\text{ideal}} \left[\begin{array}{l} -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \\ = -\sigma(\mathbf{K}(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (\mathbf{K}(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{array} \right] \\
& = \frac{1}{|\mathbb{K}|} + \frac{1}{|\mathbb{K}|} = \frac{2}{|\mathbb{K}|},
\end{aligned}$$

where the probabilities come from that (i) each $\mathbf{c}_i$ in $\mathcal{Q}_O$ is uniform and pairwise independent in the ideal world, and (ii) each $z_{i,j}$ computed from $\mathbf{c}_i$ is also uniform due to (12). Taking a union bound over all distinct pairs over $\mathcal{Q}_O \times [1, h-1]$, it holds that

$$\Pr_{\text{ideal}} [\text{bad}_1] \leq \frac{t^2(h-1)^2}{|\mathbb{K}|}. \quad (17)$$

As for bad_2 , it is sufficient to bound $\Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1]$ using the following three sub-cases. In the *first* sub-case where

$$\begin{aligned}
& \forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_O \times [1, h-1] : \\
& \left(\begin{array}{l} (\sigma(\mathbf{K}(\beta_i) - w_{i,j}), \dots) \notin \mathcal{Q}_\pi \\ \wedge (\dots, -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})) \notin \mathcal{Q}_\pi \end{array} \right),
\end{aligned}$$

it is clear that bad_2 will not happen.

Otherwise, in the following two complement sub-cases, $\mathcal{S}$ can extract the global key Δ (which is a part of the transcript $\mathcal{Q}$) by solving the offset $\mathbf{K}(\beta_i)$ from either $x = \sigma(\mathbf{K}(\beta_i) - w_{i,j})$ or $y = -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})$ according to (13) and computing $\Delta = \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i))$ for each pair in $\mathcal{Q}_O$ with $\beta_i \neq 0$. Then, $\mathcal{S}$ will use Δ to program π and π^{-1} to keep (14). If the programming succeeds, bad_2 will not happen since (14) holds for each pair in $\mathcal{Q}_O$ and each $j \in [1, h-1]$.

In the *second* sub-case where a new query to π or π^{-1} triggers the programming (i.e., query_1 case), the programming must succeed conditioned on $\neg \text{bad}_1$. The condition $\neg \text{bad}_1$ ensures that, for every $\left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_O \times [1, h-1]$, the pre-image $x := \sigma(\mathbf{K}(\beta_i) - w_{i,j})$ and the image $y := -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})$ to be programmed do not incur collision. More importantly, these two entries are not determined by the query-answer pairs in $\mathcal{Q}_\pi$ up to this time. Otherwise, $\exists(x, \dots) \in \mathcal{Q}_\pi$ or $\exists(\dots, y) \in \mathcal{Q}_\pi$ ahead of the current query such that the programming must have happened for the same $\mathcal{Q}_O$ (i.e., contradicting that the programming is triggered by the current query). Hence, all pairs in $\mathcal{Q}_O$ can be programmed as per (14). The answer to the current query is determined by the programmed result. The surely successful programming makes bad_2 impossible.

Consider the *third* sub-case where the programming arises from a new pair added to $\mathcal{Q}_O$ (i.e., query_2 case). Assume that the global key Δ (and its associated $\mathbf{K}(\beta_i) = \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$) to be used in the programming is extracted from this pair, an existing $(x^*, \dots) \in \mathcal{Q}_\pi$ or $(\dots, y^*) \in \mathcal{Q}_\pi$, and

$j^* \in [1, h-1]$. Note that either $(x^*, \dots) \in \mathcal{Q}_\pi$ or $(\dots, y^*) \in \mathcal{Q}_\pi$ has been given to $\mathcal{Z}$ and *cannot* be programmed. We bound the probability of this *undesired* sub-case. In the ideal world,

$$\begin{aligned}
& \Pr_{\text{ideal}} \left[\begin{array}{c} (x^* = \sigma(K(\beta_i) - w_{i,j^*})) \\ \vee (y^* = -\sigma(K(\beta_i) - w_{i,j^*}) + (-1)^{\mathbf{r}_i^{(j^*)}} \cdot (-\mathbf{r}_i^{(j^*)} \cdot (K(\beta_i) - w_{i,j^*}) + z_{i,j^*})) \end{array} \right] \\
& \leq \Pr_{\text{ideal}} [x^* = \sigma(K(\beta_i) - w_{i,j^*})] \\
& \quad + \Pr_{\text{ideal}} [y^* = -\sigma(K(\beta_i) - w_{i,j^*}) + (-1)^{\mathbf{r}_i^{(j^*)}} \cdot (-\mathbf{r}_i^{(j^*)} \cdot (K(\beta_i) - w_{i,j^*}) + z_{i,j^*})] \\
& = \Pr_{\text{ideal}} [w_{i,j^*} = K(\beta_i) - \sigma^{-1}(x^*)] \\
& \quad + \Pr_{\text{ideal}} [z_{i,j^*} = (-1)^{\mathbf{r}_i^{(j^*)}} \cdot (y^* + \sigma(K(\beta_i) - w_{i,j^*})) + \mathbf{r}_i^{(j^*)} \cdot (K(\beta_i) - w_{i,j^*})] \\
& = \frac{1}{|\mathbb{K}|} + \frac{1}{|\mathbb{K}|} = \frac{2}{|\mathbb{K}|},
\end{aligned}$$

where z_{i,j^*} and w_{i,j^*} are computed from the newly added pair as per (12), and the probabilities are taken over the uniform $\mathbf{c}_i$ in $\mathcal{Q}_\mathcal{O}$. Taking a union bound over $\mathcal{Q}_\pi \times \mathcal{Q}_\mathcal{O} \times [1, h-1]$ (where we assume $|\mathcal{Q}_\pi| = p = p(\lambda)$ is polynomially bounded), we can see that this sub-case happens with probability at most $2pt(h-1)/|\mathbb{K}|$.

Combining the above three sub-cases, we have that

$$\Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \leq \frac{2pt(h-1)}{|\mathbb{K}|}. \quad (18)$$

Using (17) and (18), it holds that

$$\begin{aligned}
\Pr_{\text{ideal}} [\mathcal{Q} \in \mathcal{T}_{\text{bad}}] &= \Pr_{\text{ideal}} [\text{bad}_1 \vee \text{bad}_2] \\
&= \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2] - \Pr_{\text{ideal}} [\text{bad}_1 \wedge \text{bad}_2] \\
&= \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \cdot \Pr_{\text{ideal}} [\neg \text{bad}_1] \\
&\leq \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \\
&\leq \frac{t^2(h-1)^2 + 2pt(h-1)}{|\mathbb{K}|} = \text{negl}(\lambda).
\end{aligned} \quad (19)$$

We proceed to bound the ratio of $\Pr_{\text{real}} [\mathcal{Q}]$ to $\Pr_{\text{ideal}} [\mathcal{Q}]$ for some fixed **good** transcript $\mathcal{Q} = (\Delta, \mathcal{Q}_\pi, \mathcal{Q}_\mathcal{O})$. Let $\mathcal{L} \subseteq \mathcal{Q}_\mathcal{O} \times [1, h-1]$ such that

$$\begin{aligned}
& \forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{L} : \\
& \left(\begin{array}{c} (\sigma(K(\beta_i) - w_{i,j}), \dots) \in \mathcal{Q}_\pi \\ \vee (\dots, -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j})) \in \mathcal{Q}_\pi \end{array} \right).
\end{aligned}$$

Conditioned on $\neg(\text{bad}_1 \vee \text{bad}_2)$ implicit in good transcripts, $\mathcal{L}$ induces a subset $\mathcal{Q}_\pi(\mathcal{L}) \subseteq \mathcal{Q}_\pi$. There exists a *bijective* correspondence between a $(x, y) \in \mathcal{Q}_\pi(\mathcal{L})$ and the $(z_{i,j}, w_{i,j})$ computed as per (12) from a pair in $\mathcal{L}$ such that

$$\begin{aligned}
x &= \sigma(K(\beta_i) - w_{i,j}), \\
y &= -\sigma(K(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (K(\beta_i) - w_{i,j}) + z_{i,j}).
\end{aligned}$$

It is clear that $|\mathcal{L}| = |\mathcal{Q}_\pi(\mathcal{L})| \leq \min(|\mathcal{Q}_\pi|, |\mathcal{Q}_\mathcal{O}| \cdot (h-1)) = \min(p, t(h-1))$.

For $\mathcal{X} \subseteq \mathcal{Q}_\pi$, let $\pi \sim \mathcal{X}$ denote the event that the permutation π is consistent with the queries and answers in $\mathcal{X}$, i.e., $\forall (x, y) \in \mathcal{X} : \pi(x) = y$. For $j \in [1, h]$, let $\mathbf{c}(j) \sim \mathcal{Q}_\mathcal{O}$ denote the event that

the prefix $(\mathbf{c}_i^{(0)}, \dots, \mathbf{c}_i^{(j-1)})$ in each $i \in [t]$ parallel execution in the **Initialize** phase take the literal values in $\mathcal{Q}_{\mathcal{O}}$. For $\mathcal{Y} \subseteq \mathcal{Q}_{\mathcal{O}} \times [1, h-1]$, let $(\pi, \Delta) \sim \mathcal{Y}$ denote the *real-world* event that π and the global key Δ are consistent with the pairs in $\mathcal{Y}$, i.e.,

$$\forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Y} : \\ \pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j})) = -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)}) \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j},$$

where $z_{i,j}$ and $w_{i,j}$ are computed from (12), and $\mathbf{K}(\beta_i) = \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$. For $1 \leq b \leq a$, we write the falling factorial as $(a)_b := a \cdots (a - b + 1)$.

In the real world, $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a random permutation without programming, and

$$\begin{aligned} \Pr_{\text{real}}[\mathcal{Q}] &= \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \wedge (\pi, \Delta) \sim \mathcal{Q}_{\mathcal{O}} \times [1, h-1] \mid \pi \sim \mathcal{Q}_{\pi} \wedge \Delta] \\ &\quad \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_{\pi} \wedge \Delta] \\ &= \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \wedge (\pi, \Delta) \sim \mathcal{Q}_{\mathcal{O}} \times [1, h-1] \mid \pi \sim \mathcal{Q}_{\pi} \wedge \Delta] \\ &\quad \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_{\pi}] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{Q}_{\mathcal{O}} \times [1, h-1] \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \mid \pi \sim \mathcal{Q}_{\pi} \wedge \Delta] \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_{\pi}] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[\begin{array}{l} (\pi, \Delta) \sim \mathcal{L} \\ \wedge (\pi, \Delta) \sim (\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L} \end{array} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}}] \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_{\pi}] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[(\pi, \Delta) \sim (\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \wedge (\pi, \Delta) \sim \mathcal{L} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \end{array} \right] \cdot \frac{1}{|\mathbb{K}|^t} \cdot \frac{1}{(|\mathbb{K}|)_p} \cdot \frac{1}{|\mathbb{K}|}. \end{aligned}$$

Conditioned on $\neg \text{bad}_2$ in the good transcript, the definition of $\mathcal{L}$ implies that

$$\Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \end{array} \right] = 1.$$

Then, $\neg \text{bad}_1$ ensures that, for each $\left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1]$, the permutation entries $\pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j}))$ and $\pi^{-1}(-\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)}) \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})$ are not determined by other pairs in $\mathcal{Q}_{\mathcal{O}} \times [1, h-1]$. For those pairs in $(\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L}$, these two entries are even not determined by all queries in $\mathcal{Q}_{\pi}$ due to $\neg \text{bad}_2$ and the definition of $\mathcal{L}$. By the randomness of π , we have

$$\Pr_{\text{real}} \left[(\pi, \Delta) \sim (\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \wedge (\pi, \Delta) \sim \mathcal{L} \end{array} \right] = \frac{1}{(|\mathbb{K}| - p)_{t(h-1)-|\mathcal{L}|}}.$$

Using these results, we have

$$\begin{aligned} \Pr_{\text{real}}[\mathcal{Q}] &= \frac{1}{(|\mathbb{K}| - p)_{t(h-1)-|\mathcal{L}|} \cdot (|\mathbb{K}|)_p \cdot |\mathbb{K}|^{t+1}} \\ &= \frac{1}{(|\mathbb{K}|)_{p+t(h-1)-|\mathcal{L}|} \cdot |\mathbb{K}|^{t+1}}. \end{aligned} \tag{20}$$

In the ideal world, it holds that

$$\begin{aligned}
\Pr_{\text{ideal}}[\mathcal{Q}] &= \Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_\pi \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta] \cdot \Pr_{\text{ideal}}[\mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta] \\
&= \Pr_{\text{ideal}} \left[\begin{array}{c} \pi \sim \mathcal{Q}_\pi(\mathcal{L}) \\ \wedge \pi \sim \mathcal{Q}_\pi \setminus \mathcal{Q}_\pi(\mathcal{L}) \end{array} \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta \right] \\
&\quad \cdot \Pr_{\text{ideal}}[\mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O}] \cdot \Pr_{\text{ideal}}[\Delta] \\
&= \Pr_{\text{ideal}} \left[\begin{array}{c} \pi \sim \mathcal{Q}_\pi \setminus \mathcal{Q}_\pi(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta \\ \wedge \pi \sim \mathcal{Q}_\pi(\mathcal{L}) \end{array} \right] \\
&\quad \cdot \Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_\pi(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta] \cdot \frac{1}{|\mathbb{K}|^{th} \cdot |\mathbb{K}|}.
\end{aligned}$$

By the programmed π and π^{-1} conditioned on $\neg \text{bad}_2$, the definition of $\mathcal{Q}_\pi(\mathcal{L})$ implies that

$$\begin{aligned}
\Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_\pi(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta] &= 1, \\
\Pr_{\text{ideal}} \left[\begin{array}{c} \pi \sim \mathcal{Q}_\pi \setminus \mathcal{Q}_\pi(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_\mathcal{O} \wedge \Delta \\ \wedge \pi \sim \mathcal{Q}_\pi(\mathcal{L}) \end{array} \right] &\leq \frac{1}{(|\mathbb{K}| - |\mathcal{Q}_\pi(\mathcal{L})|)_{p-|\mathcal{Q}_\pi(\mathcal{L})|}} = \frac{1}{(|\mathbb{K}| - |\mathcal{L}|)_{p-|\mathcal{L}|}}.
\end{aligned}$$

Using $|\mathcal{L}| \leq t(h-1)$, the above results, and (20), we have

$$\begin{aligned}
\Pr_{\text{ideal}}[\mathcal{Q}] &\leq \frac{1}{(|\mathbb{K}| - |\mathcal{L}|)_{p-|\mathcal{L}|}} \cdot \frac{1}{|\mathbb{K}|^{th} \cdot |\mathbb{K}|} \\
&\leq \frac{1}{(|\mathbb{K}| - t(h-1))_{p-|\mathcal{L}|}} \cdot \frac{1}{(|\mathbb{K}|)_{t(h-1)} \cdot |\mathbb{K}|^{t+1}} \\
&= \frac{1}{(|\mathbb{K}|)_{p+t(h-1)-|\mathcal{L}|} \cdot |\mathbb{K}|^{t+1}} = \Pr_{\text{real}}[\mathcal{Q}].
\end{aligned} \tag{21}$$

From (19) and (21), we can have $\epsilon_1 = \text{negl}(\lambda)$ and $\epsilon_2 = 0$. By the H-coefficient technique, $\mathcal{Z}$ can only distinguish the two worlds with a negligible advantage $\epsilon_1 + \epsilon_2$ if it is only given the transcript $\mathcal{Q} = (\Delta, \mathcal{Q}_\pi, \mathcal{Q}_\mathcal{O})$. This is exactly the advantage that $\mathcal{Z}$ can distinguish the two worlds since the output $\mathbf{K}(x)$'s in all **Extend** executions (the remaining part of the joint distribution) must be consistent with $\mathcal{Q}$ (or rather, its defined Δ and the receiver's cGGM tree) in the real world (due to the correctness of $\Pi_{\text{sPCG-sVOLE}}$) and the ideal world. In particular, when $\beta_i = 0$ for some $i \in [t]$ in the ideal world, the simulator randomly sampling $\mathbf{r}_i \leftarrow \{0, 1\}^h$ does not affect the consistency since $\beta_i = 0$ implies the i -th regular block of $\mathbf{e}$ is a zero vector so that the position where $\beta_i = 0$ is located can be arbitrary.

In conclusion, no matter whether the honest $\mathbf{P}_0$ aborts or not, the two worlds are statistically indistinguishable. Therefore, this lemma holds. $\square$

D Proof of Theorem 2

Theorem 2. *In the $\mathcal{F}_{\text{sVOLE}}$ -hybrid model and under the $\text{EA-LPN}(t, n, N, \ell, \mathbb{F})$ assumption, the protocol $\Pi_{\text{sPCG-sVOLE}}$ (Fig. 5, but removing the consistency check) UC-realizes the functionality $\mathcal{F}_{\text{sPCG-sVOLE}}$ (Fig. 4) against any semi-honest adversary, if hash function $\mathbf{H}(x) = \pi(\sigma(x)) + \sigma(x)$, where $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a (programmable) random permutation and $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an efficient linear orthomorphism.*

Proof. Without the consistency check, the view of the corrupted sender contains only $\{\mathbf{d}_i\}_i$ and the outputs of $\mathcal{F}_{\text{sVOLE}}$. These messages can be trivially simulated using random values. Moreover, the

output distribution of the sender can be consistent with the honest receiver due to the correctness of $\Pi_{\text{sPCG-sVOLE}}$. Thus, the semi-honest security against the sender holds. Note that in this case, the sender has no chance to learn the (on-average) one-bit leakage of the noise vector $\mathbf{e}$ so that the assumption $\text{EA-LPN}(t, n, N, \ell, \mathcal{R})$ suffices.

For the corrupted receiver, its view contains $\{\mathbf{c}_i\}_i$ and the outputs of $\mathcal{F}_{\text{sVOLE}}$ (and the random permutation for $\mathbf{H}$ calls in cGGM). The simulation in this case is exactly that proved in the semi-honest case of [GYW⁺23, Theorem 7, full version], implying that the semi-honest security against the receiver. It is also a simplified version of the proof of malicious security in Lemma 3 since the semi-honest receiver will not cause $\mathbf{r}_i \notin \{0, 1\}^h \subseteq \mathbb{F}^h$. The detailed proof is as follows.

When the receiver is corrupted, the simulator $\mathcal{S}$ in the ideal world emulates the random permutation π and its inverse π^{-1} throughout the lifespan of the distinguishing game. During the distinguishing game in either world, let $\mathcal{Q}_\pi$ denote an *ordered* list of the adversary's query-answer pairs of the random permutation or its inverse (note that $(x, y) \in \mathcal{Q}_\pi$ means that the adversary learns $\pi(x) = y$, regardless of whether it queried $\pi(x)$ or $\pi^{-1}(y)$), and $\mathcal{Q}_\mathcal{O}$ denote an *ordered* list of the adversary's query-answer pairs of the "oracle" emulated by the non-corrupted machine(s) (i.e., the honest $\mathbf{P}_0$ plus the subroutine functionality $\mathcal{F}_{\text{sVOLE}}$ in the real world, or the simulator $\mathcal{S}$ in the ideal world). One can think that, for each $i \in [t]$ execution in the **Initialize** phase, $\mathcal{Q}_\mathcal{O}$ records the query $(\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i))$ (i.e., the effective sVOLE output committed by the corrupted $\mathbf{P}_1$, see below) with its answer $\mathbf{c}_i$ (i.e., the transcript sent by the honest $\mathbf{P}_0$). $\mathcal{S}$ has access to $\mathcal{Q}_\pi$ and $\mathcal{Q}_\mathcal{O}$ in the ideal world.

$\mathcal{S}$ sends (init) to $\mathcal{F}_{\text{sPCG-sVOLE}}$ upon receiving the first (init) from $\mathcal{A}$ to $\mathcal{F}_{\text{sVOLE}}$, and works as follows for each $i \in [t]$ parallel execution in the **Initialize** phase:

1. Upon receiving (extend, $h + 1$) from $\mathcal{A}$ to $\mathcal{F}_{\text{sVOLE}}$, $\mathcal{S}$ waits for $\mathcal{A}$ to choose its sVOLE output $(\mathbf{s}_i, \mathbf{M}(\mathbf{s}_i))$.
2. $\mathcal{S}$ extracts $(\beta_i, \mathbf{M}(\beta_i))$, receives $\mathbf{d}_i$ from $\mathcal{A}$, and
 - (a) If $\beta_i = 0$, picks a random $\mathbf{r}_i \leftarrow \{0, 1\}^h$.
 - (b) If $\beta_i \neq 0$, extracts $\mathbf{r}_i := \beta_i^{-1} \cdot (\mathbf{d}_i - \mathbf{s}_i^{(1,h)})$. Since $\mathcal{A}$ is semi-honest, it must use $\mathbf{r}_i \in \{0, 1\}^h$.
3. $\mathcal{S}$ computes $\mathbf{M}(\mathbf{r}_i) := \mathbf{r}_i \cdot \mathbf{M}(\beta_i) - \mathbf{M}(\mathbf{s}_i^{(1,h)})$.
4. $\mathcal{S}$ sends random $\mathbf{c}_i$ to $\mathcal{A}$ and appends $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i)$ to $\mathcal{Q}_\mathcal{O}$.

In the **Extend** phase:

1. $\mathcal{S}$ uses the same randomness to sample $\mathbf{b}$ and computes $(x, \mathbf{M}(x))$ as an honest $\mathbf{P}_1$ in the protocol.
2. $\mathcal{S}$ sends (extend) and $(x, \mathbf{M}(x))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$.

Moreover, $\mathcal{S}$ invokes **Global-key queries** of $\mathcal{F}_{\text{sPCG-sVOLE}}$ in the following cases:

- **query₁**: $\mathcal{Z}$ queries the random permutation π with x or its inverse π^{-1} with y .

For each $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i) \in \mathcal{Q}_\mathcal{O}$ where $\beta_i \neq 0$:

1. $\mathcal{S}$ computes

$$\begin{aligned} \alpha_i &:= \alpha_{i,0} \dots \alpha_{i,h-1} := \bar{\mathbf{r}}_i^{(0)} \dots \bar{\mathbf{r}}_i^{(h-1)} \in \{0, 1\}^h, \\ \{z_{i,j}\}_{j \in [h]} &:= \text{cGGM.OffPath}(\alpha_i, ((-1)^{\mathbf{r}_i^{(0)}}, \dots, (-1)^{\mathbf{r}_i^{(h-1)}}) \odot (-\mathbf{M}(\mathbf{r}_i) + \mathbf{c}_i)), \quad (22) \\ \forall k \in [1, h-1] : w_{i,k} &:= \sum_{j \in [k]} z_{i,j}, \end{aligned}$$

where the macro `cGGM.OffPath`, on input a path $\alpha_i \in \{0, 1\}^h$ and h sums $\{\text{sum}_{i, \bar{\alpha}_{i,j}}^{(j)}\}_{j \in [h]}$ used in `cGGM.PuncSetup` to define an h -level correlated GGM tree except the h on-path nodes, outputs the h off-path nodes for the path α_i . This macro ensures that, for $j \in [h]$,

$$z_{i,j} = (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{M}(\mathbf{r}_i^{(j)}) + \mathbf{c}_i^{(j)}) - \text{“other } \mathbf{r}_i^{(j)}\text{-side nodes on the } j\text{-th level defined by the off-path nodes } \{z_{i,k}\}_{k \in [j]} \text{”}.$$

2. For each $j \in [1, h-1]$, $\mathcal{S}$ extracts $\mathbf{K}(\beta_i)'_1 \in \mathbb{K}$ and sends $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_1))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$ if $\mathcal{Z}$ queries π with x , or extracts $\mathbf{K}(\beta_i)'_2 \in \mathbb{K}$ and $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_2))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$ if $\mathcal{Z}$ queries π^{-1} with y , where

$$\begin{aligned} \mathbf{K}(\beta_i)'_1 &:= \sigma^{-1}(x) + w_{i,j}, \\ \mathbf{K}(\beta_i)'_2 &:= \begin{cases} \sigma^{-1}(-y + z_{i,j}) + w_{i,j} & , \text{ if } \mathbf{r}_i^{(j)} = 0 \\ \sigma'^{-1}(-y - z_{i,j}) + w_{i,j} & , \text{ if } \mathbf{r}_i^{(j)} = 1 \end{cases} \end{aligned} \quad (23)$$

Note that $\sigma'(x) := \sigma(x) - x$ is a permutation as $\sigma : \mathbb{K} \rightarrow \mathbb{K}$ is an orthomorphism, and its inverse σ'^{-1} should be well-defined.

- **query₂**: A new pair $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i)$ where $\beta_i \neq 0$ is added to $\mathcal{Q}_{\mathcal{O}}$ during execution.

From this pair, $\mathcal{S}$ computes $\{z_{i,j}\}_{j \in [h]}$ and $\{w_{i,j}\}_{j \in [1, h-1]}$ as per (22). Then, for each $(x, y) \in \mathcal{Q}_{\pi}$ and each $j \in [1, h-1]$, $\mathcal{S}$ extracts both $\mathbf{K}(\beta_i)'_1$ and $\mathbf{K}(\beta_i)'_2$ as in (23), and sends two global-key guesses $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_1))$ and $(\text{guess}, \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i)'_2))$ to $\mathcal{F}_{\text{sPCG-sVOLE}}$.

In either case, if $\mathcal{S}$ receives (**success**) from $\mathcal{F}_{\text{sPCG-sVOLE}}$ for some guess Δ , $\mathcal{S}$ will program the random permutation π and its inverse π^{-1} such that, for each $i \in [t]$, the offset $\mathbf{K}(\beta_i) := \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$, the $\{z_{i,j}\}_{j \in [h]}$ and $\{w_{i,j}\}_{j \in [1, h-1]}$ computed from the i -th $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i) \in \mathcal{Q}_{\mathcal{O}}$, and each $j \in [1, h-1]$,

$$\pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j})) = -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}), \quad (24)$$

which must hold in the real world due to the construction of $\mathbf{H}(x) = \pi(\sigma(x)) + \sigma(x)$ and the definition of $z_{i,j}$ and $w_{i,j}$. That is, the programming artificially ensures the real-world consistency among the off-path nodes on the cGGM tree in the ideal world.

Below, we use the H-coefficient technique to prove that the ideal world is statistically indistinguishable from the real one. Without loss of generality, we consider a deterministic environment $\mathcal{Z}$ and the transcript $\mathcal{Q}$, which is a part of the joint distribution of the view of $\mathcal{A}$ and the output of the honest $\mathbf{P}_0$ (note that the remaining part of this joint distribution is the $\mathbf{K}(x)$'s in all **Extend** executions, and its consistency with $\mathcal{Q}$ will be checked in the end).

A transcript $\mathcal{Q} = (\Delta, \mathcal{Q}_{\pi}, \mathcal{Q}_{\mathcal{O}})$ is **bad** if it falls into one of the following cases:

- **bad₁**. There exist two distinct transcript pairs

$$\begin{aligned} &(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j) \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1], \\ &(((\beta_{i'}, \mathbf{M}(\beta_{i'}), \mathbf{r}_{i'}, \mathbf{M}(\mathbf{r}_{i'})), \mathbf{c}_{i'}), j') \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1] \end{aligned}$$

such that, for $\mathbf{K}(\beta_i) := \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$ and $\mathbf{K}(\beta_{i'}) := \mathbf{M}(\beta_{i'}) - \beta_{i'} \cdot \Delta$, it holds that

$$\begin{aligned} &(\mathbf{K}(\beta_i) - w_{i,j} = \mathbf{K}(\beta_{i'}) - w_{i',j'}) \\ \vee &\left(\begin{aligned} &-\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \\ &= -\sigma(\mathbf{K}(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (\mathbf{K}(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{aligned} \right), \end{aligned}$$

where $z_{i,j}$, $w_{i,j}$, $z_{i',j'}$, $w_{i',j'}$ are defined as per (22). This bad case captures the collision between the queries to the random permutation and its inverse.

- **bad₂**. There exists a transcript pair

$$\left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1]$$

such that, for $\mathbf{K}(\beta_i) := \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$, it holds that

$$\begin{aligned} & \left(\begin{aligned} & (\sigma(\mathbf{K}(\beta_i) - w_{i,j}), \dots) \in \mathcal{Q}_{\pi} \\ \vee & (\dots, -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})) \in \mathcal{Q}_{\pi} \end{aligned} \right) \\ & \wedge \left(\pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j})) \neq -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \right), \end{aligned}$$

where $z_{i,j}$, $w_{i,j}$, $z_{i',j'}$, $w_{i',j'}$ are defined as per (22). This bad case captures the inconsistency in the already defined entries of the random permutation and its inverse.

Consider **bad₁** in the ideal world. For any two distinct pairs in $\mathcal{Q}_{\mathcal{O}} \times [1, h-1]$,

$$\begin{aligned} & \Pr_{\text{ideal}} \left[\begin{aligned} & (\mathbf{K}(\beta_i) - w_{i,j} = \mathbf{K}(\beta_{i'}) - w_{i',j'}) \\ \vee & \left(\begin{aligned} & -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \\ & = -\sigma(\mathbf{K}(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (\mathbf{K}(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{aligned} \right) \end{aligned} \right] \\ & \leq \Pr_{\text{ideal}} [\mathbf{K}(\beta_i) - w_{i,j} = \mathbf{K}(\beta_{i'}) - w_{i',j'}] \\ & \quad + \Pr_{\text{ideal}} \left[\begin{aligned} & -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}) \\ & = -\sigma(\mathbf{K}(\beta_{i'}) - w_{i',j'}) + (-1)^{\mathbf{r}_{i'}^{(j')}} \cdot (-\mathbf{r}_{i'}^{(j')} \cdot (\mathbf{K}(\beta_{i'}) - w_{i',j'}) + z_{i',j'}) \end{aligned} \right] \\ & = \frac{1}{|\mathbb{K}|} + \frac{1}{|\mathbb{K}|} = \frac{2}{|\mathbb{K}|}, \end{aligned}$$

where the probabilities come from that (i) each $\mathbf{c}_i$ in $\mathcal{Q}_{\mathcal{O}}$ is uniform and pairwise independent in the ideal world, and (ii) each $z_{i,j}$ computed from $\mathbf{c}_i$ is also uniform due to (22). Taking a union bound over all distinct pairs over $\mathcal{Q}_{\mathcal{O}} \times [1, h-1]$, it holds that

$$\Pr_{\text{ideal}} [\text{bad}_1] \leq \frac{t^2(h-1)^2}{|\mathbb{K}|}. \quad (25)$$

As for **bad₂**, it is sufficient to bound $\Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1]$ using the following three sub-cases. In the *first* sub-case where

$$\begin{aligned} & \forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1] : \\ & \left(\begin{aligned} & (\sigma(\mathbf{K}(\beta_i) - w_{i,j}), \dots) \notin \mathcal{Q}_{\pi} \\ \wedge & (\dots, -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})) \notin \mathcal{Q}_{\pi} \end{aligned} \right), \end{aligned}$$

it is clear that **bad₂** will not happen.

Otherwise, in the following two complement sub-cases, $\mathcal{S}$ can extract the global key Δ (which is a part of the transcript $\mathcal{Q}$) by solving the offset $\mathbf{K}(\beta_i)$ from either $x = \sigma(\mathbf{K}(\beta_i) - w_{i,j})$ or $y = -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})$ according to (23) and computing $\Delta = \beta_i^{-1} \cdot (\mathbf{M}(\beta_i) - \mathbf{K}(\beta_i))$ for each pair in $\mathcal{Q}_{\mathcal{O}}$ with $\beta_i \neq 0$. Then, $\mathcal{S}$ will use Δ to program π and

π^{-1} to keep (24). If the programming succeeds, bad_2 will not happen since (24) holds for each pair in $\mathcal{Q}_\mathcal{O}$ and each $j \in [1, h-1]$.

In the *second* sub-case where a new query to π or π^{-1} triggers the programming (i.e., query_1 case), the programming must succeed conditioned on $\neg \text{bad}_1$. The condition $\neg \text{bad}_1$ ensures that, for every $\left(\left((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i\right), j\right) \in \mathcal{Q}_\mathcal{O} \times [1, h-1]$, the pre-image $x := \sigma(\mathbf{K}(\beta_i) - w_{i,j})$ and the image $y := -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})$ to be programmed do not incur collision. More importantly, these two entries are not determined by the query-answer pairs in $\mathcal{Q}_\pi$ up to this time. Otherwise, $\exists(x, \dots) \in \mathcal{Q}_\pi$ or $\exists(\dots, y) \in \mathcal{Q}_\pi$ ahead of the current query such that the programming must have happened for the same $\mathcal{Q}_\mathcal{O}$ (i.e., contradicting that the programming is triggered by the current query). Hence, all pairs in $\mathcal{Q}_\mathcal{O}$ can be programmed as per (24). The answer to the current query is determined by the programmed result. The surely successful programming makes bad_2 impossible.

Consider the *third* sub-case where the programming arises from a new pair added to $\mathcal{Q}_\mathcal{O}$ (i.e., query_2 case). Assume that the global key Δ (and its associated $\mathbf{K}(\beta_i) = \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$) to be used in the programming is extracted from this pair, an existing $(x^*, \dots) \in \mathcal{Q}_\pi$ or $(\dots, y^*) \in \mathcal{Q}_\pi$, and $j^* \in [1, h-1]$. Note that either $(x^*, \dots) \in \mathcal{Q}_\pi$ or $(\dots, y^*) \in \mathcal{Q}_\pi$ has been given to $\mathcal{Z}$ and *cannot* be programmed. We bound the probability of this *undesired* sub-case. In the ideal world,

$$\begin{aligned} & \Pr_{\text{ideal}} \left[\bigvee \left(\begin{aligned} & (x^* = \sigma(\mathbf{K}(\beta_i) - w_{i,j^*})) \\ & (y^* = -\sigma(\mathbf{K}(\beta_i) - w_{i,j^*}) + (-1)^{\mathbf{r}_i^{(j^*)}} \cdot (-\mathbf{r}_i^{(j^*)} \cdot (\mathbf{K}(\beta_i) - w_{i,j^*}) + z_{i,j^*})) \end{aligned} \right) \right] \\ & \leq \Pr_{\text{ideal}} [x^* = \sigma(\mathbf{K}(\beta_i) - w_{i,j^*})] \\ & \quad + \Pr_{\text{ideal}} [y^* = -\sigma(\mathbf{K}(\beta_i) - w_{i,j^*}) + (-1)^{\mathbf{r}_i^{(j^*)}} \cdot (-\mathbf{r}_i^{(j^*)} \cdot (\mathbf{K}(\beta_i) - w_{i,j^*}) + z_{i,j^*})] \\ & = \Pr_{\text{ideal}} [w_{i,j^*} = \mathbf{K}(\beta_i) - \sigma^{-1}(x^*)] \\ & \quad + \Pr_{\text{ideal}} \left[z_{i,j^*} = (-1)^{\mathbf{r}_i^{(j^*)}} \cdot \left(y^* + \sigma(\mathbf{K}(\beta_i) - w_{i,j^*}) \right) + \mathbf{r}_i^{(j^*)} \cdot (\mathbf{K}(\beta_i) - w_{i,j^*}) \right] \\ & = \frac{1}{|\mathbb{K}|} + \frac{1}{|\mathbb{K}|} = \frac{2}{|\mathbb{K}|}, \end{aligned}$$

where z_{i,j^*} and w_{i,j^*} are computed from the newly added pair as per (22), and the probabilities are taken over the uniform $\mathbf{c}_i$ in $\mathcal{Q}_\mathcal{O}$. Taking a union bound over $\mathcal{Q}_\pi \times \mathcal{Q}_\mathcal{O} \times [1, h-1]$ (where we assume $|\mathcal{Q}_\pi| = p = p(\lambda)$ is polynomially bounded), we can see that this sub-case happens with probability at most $2pt(h-1)/|\mathbb{K}|$.

Combining the above three sub-cases, we have that

$$\Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \leq \frac{2pt(h-1)}{|\mathbb{K}|}. \quad (26)$$

Using (25) and (26), it holds that

$$\begin{aligned} \Pr_{\text{ideal}} [\mathcal{Q} \in \mathcal{T}_{\text{bad}}] &= \Pr_{\text{ideal}} [\text{bad}_1 \vee \text{bad}_2] \\ &= \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2] - \Pr_{\text{ideal}} [\text{bad}_1 \wedge \text{bad}_2] \\ &= \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \cdot \Pr_{\text{ideal}} [\neg \text{bad}_1] \\ &\leq \Pr_{\text{ideal}} [\text{bad}_1] + \Pr_{\text{ideal}} [\text{bad}_2 \mid \neg \text{bad}_1] \\ &\leq \frac{t^2(h-1)^2 + 2pt(h-1)}{|\mathbb{K}|} = \text{negl}(\lambda). \end{aligned} \quad (27)$$

We proceed to bound the ratio of $\Pr_{\text{real}} [\mathcal{Q}]$ to $\Pr_{\text{ideal}} [\mathcal{Q}]$ for some fixed **good** transcript $\mathcal{Q} =$

$(\Delta, \mathcal{Q}_\pi, \mathcal{Q}_\mathcal{O})$. Let $\mathcal{L} \subseteq \mathcal{Q}_\mathcal{O} \times [1, h-1]$ such that

$$\forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{L} : \\ \left(\begin{array}{l} (\sigma(\mathbf{K}(\beta_i) - w_{i,j}), \dots) \in \mathcal{Q}_\pi \\ \vee (\dots, -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j})) \in \mathcal{Q}_\pi \end{array} \right).$$

Conditioned on $\neg(\text{bad}_1 \vee \text{bad}_2)$ implicit in good transcripts, $\mathcal{L}$ induces a subset $\mathcal{Q}_\pi(\mathcal{L}) \subseteq \mathcal{Q}_\pi$. There exists a *bijective* correspondence between a $(x, y) \in \mathcal{Q}_\pi(\mathcal{L})$ and the $(z_{i,j}, w_{i,j})$ computed as per (22) from a pair in $\mathcal{L}$ such that

$$\begin{aligned} x &= \sigma(\mathbf{K}(\beta_i) - w_{i,j}), \\ y &= -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}). \end{aligned}$$

It is clear that $|\mathcal{L}| = |\mathcal{Q}_\pi(\mathcal{L})| \leq \min(|\mathcal{Q}_\pi|, |\mathcal{Q}_\mathcal{O}| \cdot (h-1)) = \min(p, t(h-1))$.

For $\mathcal{X} \subseteq \mathcal{Q}_\pi$, let $\pi \sim \mathcal{X}$ denote the event that the permutation π is consistent with the queries and answers in $\mathcal{X}$, i.e., $\forall (x, y) \in \mathcal{X} : \pi(x) = y$. For $j \in [1, h]$, let $\mathbf{c}(j) \sim \mathcal{Q}_\mathcal{O}$ denote the event that the prefix $(\mathbf{c}_i^{(0)}, \dots, \mathbf{c}_i^{(j-1)})$ in each $i \in [t]$ parallel execution in the **Initialize** phase take the literal values in $\mathcal{Q}_\mathcal{O}$. For $\mathcal{Y} \subseteq \mathcal{Q}_\mathcal{O} \times [1, h-1]$, let $(\pi, \Delta) \sim \mathcal{Y}$ denote the *real-world* event that π and the global key Δ are consistent with the pairs in $\mathcal{Y}$, i.e.,

$$\begin{aligned} \forall \left(((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i), j \right) \in \mathcal{Y} : \\ \pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j})) = -\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}), \end{aligned}$$

where $z_{i,j}$ and $w_{i,j}$ are computed from (22), and $\mathbf{K}(\beta_i) = \mathbf{M}(\beta_i) - \beta_i \cdot \Delta$. For $1 \leq b \leq a$, we write the falling factorial as $(a)_b := a \cdots (a - b + 1)$.

In the real world, $\pi : \mathbb{K} \rightarrow \mathbb{K}$ is a random permutation without programming, and

$$\begin{aligned} \Pr_{\text{real}}[\mathcal{Q}] &= \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \wedge (\pi, \Delta) \sim \mathcal{Q}_\mathcal{O} \times [1, h-1] \mid \pi \sim \mathcal{Q}_\pi \wedge \Delta] \\ &\quad \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_\pi \wedge \Delta] \\ &= \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \wedge (\pi, \Delta) \sim \mathcal{Q}_\mathcal{O} \times [1, h-1] \mid \pi \sim \mathcal{Q}_\pi \wedge \Delta] \\ &\quad \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_\pi] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{Q}_\mathcal{O} \times [1, h-1] \mid \begin{array}{l} \pi \sim \mathcal{Q}_\pi \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \mid \pi \sim \mathcal{Q}_\pi \wedge \Delta] \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_\pi] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[\begin{array}{l} (\pi, \Delta) \sim \mathcal{L} \\ \wedge (\pi, \Delta) \sim (\mathcal{Q}_\mathcal{O} \times [1, h-1]) \setminus \mathcal{L} \end{array} \mid \begin{array}{l} \pi \sim \mathcal{Q}_\pi \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}}[\mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O}] \cdot \Pr_{\text{real}}[\pi \sim \mathcal{Q}_\pi] \cdot \Pr_{\text{real}}[\Delta] \\ &= \Pr_{\text{real}} \left[(\pi, \Delta) \sim (\mathcal{Q}_\mathcal{O} \times [1, h-1]) \setminus \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_\pi \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \wedge (\pi, \Delta) \sim \mathcal{L} \end{array} \right] \\ &\quad \cdot \Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_\pi \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \end{array} \right] \cdot \frac{1}{|\mathbb{K}|^t} \cdot \frac{1}{(|\mathbb{K}|)_p} \cdot \frac{1}{|\mathbb{K}|}. \end{aligned}$$

Conditioned on $\neg \text{bad}_2$ in the good transcript, the definition of $\mathcal{L}$ implies that

$$\Pr_{\text{real}} \left[(\pi, \Delta) \sim \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_\pi \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_\mathcal{O} \end{array} \right] = 1.$$

Then, $\neg\text{bad}_1$ ensures that, for each $((\beta_i, \mathbf{M}(\beta_i), \mathbf{r}_i, \mathbf{M}(\mathbf{r}_i)), \mathbf{c}_i, j) \in \mathcal{Q}_{\mathcal{O}} \times [1, h-1]$, the permutation entries $\pi(\sigma(\mathbf{K}(\beta_i) - w_{i,j}))$ and $\pi^{-1}(-\sigma(\mathbf{K}(\beta_i) - w_{i,j}) + (-1)^{\mathbf{r}_i^{(j)}} \cdot (-\mathbf{r}_i^{(j)} \cdot (\mathbf{K}(\beta_i) - w_{i,j}) + z_{i,j}))$ are not determined by other pairs in $\mathcal{Q}_{\mathcal{O}} \times [1, h-1]$. For those pairs in $(\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L}$, these two entries are even not determined by all queries in $\mathcal{Q}_{\pi}$ due to $\neg\text{bad}_2$ and the definition of $\mathcal{L}$. By the randomness of π , we have

$$\Pr_{\text{real}} \left[(\pi, \Delta) \sim (\mathcal{Q}_{\mathcal{O}} \times [1, h-1]) \setminus \mathcal{L} \mid \begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \wedge \Delta \\ \wedge \mathbf{c}(1) \sim \mathcal{Q}_{\mathcal{O}} \wedge (\pi, \Delta) \sim \mathcal{L} \end{array} \right] = \frac{1}{(|\mathbb{K}| - p)_{t(h-1)-|\mathcal{L}|}}.$$

Using these results, we have

$$\begin{aligned} \Pr_{\text{real}}[\mathcal{Q}] &= \frac{1}{(|\mathbb{K}| - p)_{t(h-1)-|\mathcal{L}|} \cdot (|\mathbb{K}|)_p \cdot |\mathbb{K}|^{t+1}} \\ &= \frac{1}{(|\mathbb{K}|)_{p+t(h-1)-|\mathcal{L}|} \cdot |\mathbb{K}|^{t+1}}. \end{aligned} \quad (28)$$

In the ideal world, it holds that

$$\begin{aligned} \Pr_{\text{ideal}}[\mathcal{Q}] &= \Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_{\pi} \mid \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta] \cdot \Pr_{\text{ideal}}[\mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta] \\ &= \Pr_{\text{ideal}} \left[\begin{array}{l} \pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \\ \wedge \pi \sim \mathcal{Q}_{\pi} \setminus \mathcal{Q}_{\pi}(\mathcal{L}) \end{array} \mid \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta \right] \\ &\quad \cdot \Pr_{\text{ideal}}[\mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}}] \cdot \Pr_{\text{ideal}}[\Delta] \\ &= \Pr_{\text{ideal}} \left[\begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \setminus \mathcal{Q}_{\pi}(\mathcal{L}) \\ \wedge \pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \end{array} \mid \begin{array}{l} \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta \\ \wedge \pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \end{array} \right] \\ &\quad \cdot \Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta] \cdot \frac{1}{|\mathbb{K}|^{th} \cdot |\mathbb{K}|}. \end{aligned}$$

By the programmed π and π^{-1} conditioned on $\neg\text{bad}_2$, the definition of $\mathcal{Q}_{\pi}(\mathcal{L})$ implies that

$$\begin{aligned} \Pr_{\text{ideal}}[\pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \mid \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta] &= 1, \\ \Pr_{\text{ideal}} \left[\begin{array}{l} \pi \sim \mathcal{Q}_{\pi} \setminus \mathcal{Q}_{\pi}(\mathcal{L}) \\ \wedge \pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \end{array} \mid \begin{array}{l} \mathbf{c}(h) \sim \mathcal{Q}_{\mathcal{O}} \wedge \Delta \\ \wedge \pi \sim \mathcal{Q}_{\pi}(\mathcal{L}) \end{array} \right] &\leq \frac{1}{(|\mathbb{K}| - |\mathcal{Q}_{\pi}(\mathcal{L})|)_{p-|\mathcal{Q}_{\pi}(\mathcal{L})|}} = \frac{1}{(|\mathbb{K}| - |\mathcal{L}|)_{p-|\mathcal{L}|}}. \end{aligned}$$

Using $|\mathcal{L}| \leq t(h-1)$, the above results, and (28), we have

$$\begin{aligned} \Pr_{\text{ideal}}[\mathcal{Q}] &\leq \frac{1}{(|\mathbb{K}| - |\mathcal{L}|)_{p-|\mathcal{L}|}} \cdot \frac{1}{|\mathbb{K}|^{th} \cdot |\mathbb{K}|} \\ &\leq \frac{1}{(|\mathbb{K}| - t(h-1))_{p-|\mathcal{L}|}} \cdot \frac{1}{(|\mathbb{K}|)_{t(h-1)} \cdot |\mathbb{K}|^{t+1}} \\ &= \frac{1}{(|\mathbb{K}|)_{p+t(h-1)-|\mathcal{L}|} \cdot |\mathbb{K}|^{t+1}} = \Pr_{\text{real}}[\mathcal{Q}]. \end{aligned} \quad (29)$$

From (27) and (29), we can have $\epsilon_1 = \text{negl}(\lambda)$ and $\epsilon_2 = 0$. By the H-coefficient technique, $\mathcal{Z}$ can only distinguish the two worlds with a negligible advantage $\epsilon_1 + \epsilon_2$ if it is only given the transcript $\mathcal{Q} = (\Delta, \mathcal{Q}_{\pi}, \mathcal{Q}_{\mathcal{O}})$. This is exactly the advantage that $\mathcal{Z}$ can distinguish the two worlds since the output $\mathbf{K}(x)$'s in all **Extend** executions (the remaining part of the joint distribution) must be consistent with $\mathcal{Q}$ (or rather, its defined Δ and the receiver's cGGM tree) in the real world (due to the correctness of $\Pi_{\text{sPCG-sVOLE}}$) and the ideal world. In particular, when $\beta_i = 0$ for some $i \in [t]$

in the ideal world, the simulator randomly sampling $\mathbf{r}_i \leftarrow \{0, 1\}^h$ does not affect the consistency since $\beta_i = 0$ implies the i -th regular block of $\mathbf{e}$ is a zero vector so that the position where $\beta_i = 0$ is located can be arbitrary. Therefore, the security against the semi-honest receiver holds.

Putting the two cases together, this theorem holds. $\square$